

**IMPLEMENTASI FINE-TUNING OPENAI WHISPER SMALL
UNTUK AUTOMATIC SPEECH RECOGNITION BAHASA
MINANGKABAU**

TUGAS AKHIR

Diajukan sebagai syarat menyelesaikan jenjang strata Satu (S-1) di
Program Studi Teknik Informatika, Fakultas Teknologi Industri, Institut
Teknologi Sumatera

Oleh:

Fathan Andi Kartagama

122140055



**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN
2025**

LEMBAR PENGESAHAN

Saya menyatakan bahwa Tugas Akhir berjudul "Implementasi Fine-Tuning OpenAI Whisper Small untuk Automatic Speech Recognition Bahasa Minangkabau" merupakan hasil karya saya sendiri dan belum pernah diajukan, baik sebagian maupun seluruhnya, di Institut Teknologi Sumatera atau institusi pendidikan lain oleh saya maupun pihak lain.

Lampung Selatan, 29 November 2025
Penulis,

Fathan Andi Kartagama
NIM. 122140055

Foto 2x3

Diperiksa dan disetujui oleh,
Pembimbing

1. Martin Clinton Tosima Manullang, Ph.D.
19930109 201903 1 017

.....

Penguji

1. Dosen Penguji I
NIP. 19900000 2000 00 0 000
2. Dosen Penguji II
NIP. 19900000 2000 00 0 000

.....

.....

Disahkan oleh,
Koordinator Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Andika Setiawan, S.Kom., M.Cs.
NIP. 19911127 2022 03 1 007

HALAMAN PERNYATAAN ORISINALITAS

Tugas Akhir dengan judul “Implementasi Fine-Tuning OpenAI Whisper Small untuk Automatic Speech Recognition Bahasa Minangkabau” adalah karya saya sendiri, dan semua sumber baik yang dikutip maupun dirujuk telah saya nyatakan benar.

Nama : Fathan Andi Kartagama

NIM : 122140055

Tanda Tangan :

Tanggal : 29-11-2025

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Institut Teknologi Sumatera, saya yang bertanda tangan di bawah ini:

Nama : Fathan Andi Kartagama

NIM : 122140055

Program Studi : Teknik Informatika

Fakultas : Teknologi Industri

Jenis Karya : Tugas Akhir

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Institut Teknologi Sumatera **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Implementasi Fine-Tuning OpenAI Whisper Small untuk Automatic Speech Recognition Bahasa Minangkabau

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Institut Teknologi Sumatera berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Lampung Selatan
Pada tanggal : 29 November 2025

Yang menyatakan

Fathan Andi Kartagama

KATA PENGANTAR

Pada halaman ini mahasiswa berkesempatan untuk menyatakan terima kasih secara tertulis kepada pembimbing dan pihak lain yang telah memberi bimbingan, nasihat, saran dan kritik, kepada mereka yang telah membantu melakukan penelitian, kepada perorangan atau lembaga yang telah memberi bantuan keuangan, materi dan/atau sarana. Cara menulis kata pengantar beraneka ragam, tetapi hendaknya menggunakan kalimat yang baku. Ucapan terima kasih agar dibuat tidak berlebihan dan dibatasi pada pihak yang terkait secara ilmiah (berhubungan dengan subjek/materi penelitian).

Puji syukur kehadiran Allah SWT/Tuhan Yang Maha Esa atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga penyusunan tugas akhir ini telah terselesaikan dengan baik. Dalam penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. [Rektor ITERA] selaku Rektor Institut Teknologi Sumatera.
2. [Dekan FTI] selaku Dekan Fakultas Teknologi Industri.
3. [Koor Prodi IF] selaku Ketua Program Studi Teknik Informatika.
4. [Dosen Pembimbing] selaku Dosen Pembimbing atas ide, waktu, tenaga, perhatian, dan masukan yang telah disumbangsihkan kepada penulis.
5. [Isi nama lainnya]

Akhir kata penulis berharap semoga tugas akhir ini dapat memberikan manfaat bagi kita semua.

RINGKASAN

Implementasi Fine-Tuning OpenAI Whisper Small untuk Automatic Speech

Recognition Bahasa Minangkabau

Fathan Andi Kartagama

Halaman Ringkasan berisi uraian singkat tentang latar belakang masalah, rumusan masalah, tujuan, metodologi penelitian, hasil dan analisis data, serta kesimpulan dan saran. Isi ringkasan tidak lebih dari 1000 kata (sekitar maksimal 2 halaman).

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

ABSTRAK

Implementasi Fine-Tuning OpenAI Whisper Small untuk Automatic Speech

Recognition Bahasa Minangkabau

Fathan Andi Kartagama

Halaman ABSTRAK berisi uraian tentang latar belakang, tujuan, metodologi penelitian, hasil / kesimpulan. Ditulis dalam BAHASA INDONESIA tidak lebih dari 250 kata, dengan jarak antar baris satu spasi. Pada akhir abstrak ditulis kata “Kata Kunci” yang dicetak tebal, diikuti tanda titik dua dan kata kunci yang tidak lebih dari 5 kata. Kata kunci terdiri dari kata-kata yang khusus menunjukkan dan berkaitan dengan bahan yang diteliti, metode/instrumen yang digunakan, topik penelitian. Kata kunci diketik pada jarak dua spasi dari baris akhir isi abstrak.

Kata Kunci: kunci1, kunci2

ABSTRACT

Thesis Title With ABC Method (Case Study: XYZ)

Fathan Andi Kartagama

Halaman ABSTRACT berisi uraian tentang latar belakang, tujuan, metodologi penelitian, hasil / kesimpulan. Ditulis dalam BAHASA INGGRIS tidak lebih dari 250 kata, dengan jarak antar baris satu spasi. Secara khusus, kata dan kalimat pada halaman ini tidak perlu ditulis dengan huruf miring meskipun menggunakan Bahasa Inggris, kecuali terdapat huruf asing lain yang ditulis dengan huruf miring (misalnya huruf Latin atau Greek, dll). Pada akhir abstract ditulis kata “Keywords” yang dicetak tebal, diikuti tanda titik dua dan kata kunci yang tidak lebih dari 5 kata. Keywords terdiri dari kata-kata yang khusus menunjukkan dan berkaitan dengan bahan yang diteliti, metode/instrumen yang digunakan, topik penelitian. Keywords diketik pada jarak dua spasi dari baris akhir isi abstrak.

Keywords: keywords1, keywords2

DAFTAR ISI

LEMBAR PENGESAHAN	ii
HALAMAN PERNYATAAN ORISINALITAS	iii
HALAMAN PERSETUJUAN PUBLIKASI	iv
KATA PENGANTAR	v
RINGKASAN	vi
ABSTRAK	vii
ABSTRACT	viii
DAFTAR ISI	ix
DAFTAR TABEL	xiii
DAFTAR GAMBAR	xiv
DAFTAR RUMUS	xv
DAFTAR KODE	xvi
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	5
1.4 Batasan Masalah	6
1.5 Manfaat Penelitian	7
1.6 Sistematika Penulisan	9
BAB II TINJAUAN PUSTAKA	11
2.1 Tinjauan Pustaka	11
2.1.1 Model Whisper dan Fondasi Teoritis	14

2.1.2	Strategi Fine-Tuning untuk Bahasa Low-Resource	15
2.1.3	Adaptasi untuk Aplikasi Dunia Nyata	16
2.1.4	Optimisasi Proses Fine-Tuning	16
2.1.5	Transfer Learning Multibahasa	17
2.1.6	ASR untuk Bahasa dengan Data Sangat Terbatas	18
2.1.7	Sintesis dan Posisi Penelitian	18
2.2	Dasar Teori	19
2.2.1	Automatic Speech Recognition (ASR)	19
2.2.1.1	Bahasa Low-Resource dan Tantangannya	20
2.2.2	Transfer Learning dan Model Pre-Trained	20
2.2.2.1	Model Pre-Trained untuk ASR	21
2.2.3	Whisper dan Arsitekturnya	21
2.2.3.1	Varian Model dan Parameter	22
2.2.3.2	Encoder	23
2.2.3.3	Decoder	24
2.2.3.4	Whisper Small	24
2.2.4	Fine-Tuning: Konsep dan Strategi	25
2.2.4.1	Full Fine-Tuning	25
2.2.4.2	Freeze Encoder Fine-Tuning	25
2.2.4.3	Parameter-Efficient Fine-Tuning (PEFT)	26
2.2.4.4	LoRA (Low-Rank Adaptation)	27
2.2.5	Metrik Evaluasi ASR	28
2.2.5.1	Word Error Rate (WER)	28
2.2.5.2	Character Error Rate (CER)	28
2.2.6	Data Preprocessing untuk ASR	29
2.2.6.1	Audio Preprocessing	29
2.2.6.2	Text Preprocessing	30
2.2.6.3	Data Augmentation	30
2.2.7	Regularisasi dan Optimisasi	31

2.2.7.1	Regularisasi	31
2.2.7.2	Optimisasi	31
2.2.8	K-Fold Cross-Validation	31
2.2.8.1	Prinsip Dasar K-Fold Cross-Validation	32
2.2.8.2	Keuntungan untuk Dataset Kecil	32
2.2.8.3	Pemilihan Nilai K	33
2.2.8.4	Stratified K-Fold	33
2.2.8.5	Statistical Testing dengan Cross-Validation	33
BAB III METODE PENELITIAN		34
3.1	Alur Penelitian	34
3.2	Penjabaran Langkah Penelitian	36
3.2.1	Identifikasi Masalah	36
3.2.2	Studi Literatur	37
3.2.3	Pengumpulan Dataset	39
3.2.4	Preprocessing Dataset	41
3.2.4.1	Audio Preprocessing	41
3.2.4.2	Text Preprocessing	43
3.2.4.3	5-Fold Cross-Validation	44
3.2.5	Konfigurasi Model Whisper Small	45
3.2.6	Implementasi Fine-Tuning	46
3.2.6.1	Freeze Encoder Fine-Tuning	47
3.2.6.2	LoRA Fine-Tuning	48
3.2.6.3	Regularization	49
3.2.7	Training dan Monitoring	49
3.2.8	Evaluasi Model	50
3.2.9	Analisis Hasil dan Perbandingan	52
3.3	Alat dan Bahan Tugas Akhir	53
3.3.1	Alat	53

3.3.1.1	Perangkat Keras	53
3.3.1.2	Perangkat Lunak	54
3.3.2	Bahan	55
3.4	Metode Pengembangan	56
3.4.1	Pendekatan Penelitian	57
3.4.2	Alur Pengembangan	57
3.4.3	Metodologi Fine-Tuning	59
3.4.3.1	Freeze Encoder Fine-Tuning Methodology	59
3.4.3.2	LoRA Fine-Tuning Methodology	59
3.4.3.3	Aggregation dan Statistical Testing	60
3.4.4	Metode Pengujian	60
3.4.4.1	Functional Testing	60
3.4.4.2	Performance Testing	61
3.4.4.3	Comparative Testing	61
3.5	Ilustrasi Perhitungan Metode	61
3.5.1	Perhitungan Word Error Rate (WER)	61
3.5.1.1	Formula WER	61
3.5.1.2	Contoh Perhitungan	62
3.5.2	Perhitungan Character Error Rate (CER)	62
3.5.2.1	Formula CER	62
3.5.2.2	Contoh Perhitungan	63
3.5.3	Perhitungan LoRA Parameters	63
3.5.3.1	Formula LoRA Parameters	63
3.5.3.2	Contoh Perhitungan untuk Whisper Small	63
BAB IV	HASIL DAN PEMBAHASAN	65
4.1	Hasil Penelitian	65
4.2	Hasil Pengujian	65
4.3	Analisis Hasil Penelitian	66

BAB V KESIMPULAN DAN SARAN	68
5.1 Kesimpulan.....	68
5.2 Saran	68
DAFTAR PUSTAKA.....	69
LAMPIRAN.....	72
A Dataset.....	72
B Hasil Wawancara.....	72
C Rincian Kasus Uji.....	72

DAFTAR TABEL

Tabel 2.1	Ringkasan Penelitian Terdahulu (2022–2025).....	11
Tabel 2.2	Spesifikasi dan Kebutuhan Sumber Daya Varian Model Whisper	23
Tabel 3.1	Alignment untuk perhitungan WER (C=Correct, S=Substitution, D=Deletion).....	62
Tabel 4.1	Data <i>dummy</i> Pengujian.....	66

DAFTAR GAMBAR

Gambar 2.1	Arsitektur dan <i>Multitask Training Format</i> Whisper	22
Gambar 3.1	Alur Penelitian Fine-Tuning Whisper untuk ASR Bahasa Minangkabau	35
Gambar 3.2	Alur Pengembangan Sistem	58
Gambar 4.1	Contoh Graf Pengujian	66

DAFTAR RUMUS

Rumus 2.1	Formula LoRA (Low-Rank Adaptation)	27
Rumus 2.2	Formula WER (Word Error Rate)	28
Rumus 3.1	Definisi WER (Word Error Rate)	61
Rumus 3.2	Contoh Perhitungan WER	62
Rumus 3.3	Definisi CER (Character Error Rate)	62
Rumus 3.4	Contoh Perhitungan CER	63
Rumus 3.5	Formula Parameter LoRA	63
Rumus 3.6	Parameter LoRA Per Projection	64
Rumus 3.7	Parameter LoRA Per Layer	64
Rumus 3.8	Total Parameter LoRA	64
Rumus 3.9	Persentase Parameter LoRA Terhadap Total	64

DAFTAR KODE

Kode 4.1 Akuisisi Gambar 65

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam beberapa dekade terakhir, kemajuan dalam pemrosesan suara dan pengenalan ucapan (*Automatic Speech Recognition / ASR*) telah menjadi salah satu bidang riset yang sangat penting, terutama karena potensinya besar di banyak aspek kehidupan manusia, misalnya asisten suara (*Siri, Google Assistant*), sistem transkripsi otomatis, aksesibilitas bagi penyandang disabilitas, layanan publik, pendidikan, serta interaksi manusia-komputer. Model-model berbasis pembelajaran mendalam (*deep learning*) yang menggunakan arsitektur *Transformer*, atau model "*end-to-end*", semakin banyak digunakan untuk menggantikan pendekatan tradisional seperti *Hidden Markov Model* dan *Gaussian Mixture Model*.

Namun, performa ASR sangat bergantung pada ketersediaan data berbicara (*speech*) dan transkripsi yang berkualitas dan representatif dari bahasa atau dialek target. Untuk bahasa mayor seperti Inggris, Mandarin, Spanyol, data sangat melimpah; sementara untuk bahasa lokal atau dialek (termasuk bahasa daerah di Indonesia) data seringkali sangat terbatas (*low-resource*). Model yang telah dilatih pada data multibahasa besar seperti Whisper kadang masih kurang optimal ketika menghadapi dialek atau bahasa yang kurang terwakili dalam data latihannya.

OpenAI memperkenalkan model Whisper sebagai model *speech-to-text* multibahasa dengan pelatihan lemah-supervisi skala besar (*Large-Scale Weak Supervision*) (sekitar 680.000 jam data audio transkrip) untuk generalisasi ke berbagai kondisi dan bahasa [1]. Meskipun demikian, model dasar ("*base Whisper*") seringkali tidak menghasilkan akurasi memadai ketika dihadapkan pada bahasa atau dialek yang sangat spesifik atau dengan variasi fonetik tinggi

yang belum banyak direpresentasikan. Oleh karena itu, teknik *fine-tuning* terhadap model dasar menjadi sangat penting agar model "teradaptasi" ke karakteristik bahasa lokal tersebut.

Teknik *fine-tuning* Whisper telah dieksplorasi dalam sejumlah studi, terutama untuk bahasa *low-resource*. Misalnya, penelitian *Exploration of Whisper fine-tuning strategies for low-resource ASR* menunjukkan bahwa berbagai strategi *fine-tuning* dapat secara signifikan meningkatkan performa pengenalan suara untuk bahasa-bahasa dengan data terbatas [2]. Ada pula penelitian "*Fine-tuning Whisper on Low-Resource Languages for Real-World Applications*" yang memperkenalkan metode transformasi dataset kalimat-ke-audio panjang untuk menjaga kemampuan segmentasi dan bagus bagi aplikasi nyata [3].

Studi lain mengusulkan metode *fine-tuning* efisien, seperti *LoRA-Whisper*, yang bertujuan agar adaptasi ke bahasa baru bisa dilakukan tanpa menurunkan performa bahasa lain dan dengan *overhead* parameter yang lebih kecil [4]. Juga ada penelitian *Multistage Fine-tuning Strategies for Automatic Speech Recognition in Low-resource Languages* yang menggunakan strategi *multistage* (bertahap) untuk mengadaptasi model melalui bahasa yang punya kemiripan linguistik [5].

Begitu juga, studi aplikasi domain khusus (misalnya dalam konteks medis) menekankan bahwa *fine-tuning* model Whisper pada domain spesifik (dengan kosakata dan pola suara khusus) mampu meningkatkan akurasi dibandingkan model standar [6]. Dengan demikian, adaptasi model ASR (melalui *fine-tuning*) telah menjadi pendekatan penting untuk menjembatani kesenjangan antara model umum dan kebutuhan lokal atau domain khusus.

Di Indonesia, keragaman bahasa dan dialek sangat tinggi, dan banyak komunitas menggunakan bahasa daerah dalam kehidupan sehari-hari. Bahasa Minangkabau adalah salah satu bahasa daerah yang memiliki nilai budaya, sosial, dan historis tinggi, khususnya di wilayah Sumatera Barat dan daerah-

daerah diaspora Minangkabau. Namun, dalam literatur riset ASR Indonesia, fokus utama lebih banyak pada bahasa Indonesia (standar) atau beberapa dialek besar (Sunda, Jawa, dsb.), sedangkan kontribusi penelitian terhadap bahasa Minangkabau relatif sedikit.

Karena bahasa Minangkabau memiliki fonetik, kosakata, dan intonasi yang khas, transkripsi otomatis dengan model umum (bahasa Indonesia atau multibahasa) cenderung menghasilkan kesalahan (*error*) yang cukup tinggi. Hal ini dapat disebabkan oleh kekurangan data latih (audio + transkripsi) dalam bahasa Minangkabau, serta ketidakmampuan model awal menangani variasi dialek, aksen lokal, dan fenomena fonologi spesifik. Oleh karena itu, tugas untuk mengembangkan sistem ASR yang mampu mengenali ucapan bahasa Minangkabau dengan baik memerlukan adaptasi model melalui *fine-tuning*, agar model "belajar" karakteristik suara, kosakata, dan pola ucapan lokal tersebut.

Urgensi pengembangan sistem ASR untuk bahasa Minangkabau semakin meningkat seiring dengan kebutuhan nyata di berbagai sektor. Di bidang **pelestarian budaya**, terdapat banyak arsip audio sastra lisan Minangkabau (seperti *kaba*, *pantun*, dan cerita rakyat) yang tersimpan dalam format analog atau rekaman digital tanpa transkripsi, sehingga sulit diakses dan dianalisis oleh generasi muda maupun peneliti. Sistem ASR yang akurat dapat mempercepat **digitalisasi dan dokumentasi** warisan budaya ini. Di sektor **media dan konten digital**, para *content creator* lokal yang memproduksi konten video atau podcast berbahasa Minangkabau memerlukan alat transkripsi otomatis untuk membuat subtitle atau transkrip guna meningkatkan aksesibilitas dan jangkauan audiens. Selain itu, di bidang **pariwisata dan layanan publik**, sistem ASR dapat diintegrasikan ke dalam aplikasi pemandu wisata interaktif atau layanan informasi berbasis suara untuk wisatawan yang ingin memahami budaya Minangkabau. Dengan demikian, keberadaan sistem ASR bahasa Minangkabau bukan hanya bersifat "*nice to have*" untuk riset akademis, tetapi merupakan kebutuhan *essential* yang memiliki dampak praktis langsung terhadap

pelestarian budaya, ekonomi kreatif, dan aksesibilitas informasi bagi masyarakat Minangkabau.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka permasalahan utama dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana mengadaptasi (*fine-tuning*) model **OpenAI Whisper Small** agar mampu mengenali dan mentranskripsi ucapan dalam **bahasa Minangkabau** secara akurat, mengingat keterbatasan data suara dan teks (*low-resource*) yang tersedia?
2. Bagaimana proses perancangan dan implementasi sistem **Automatic Speech Recognition (ASR)** berbasis hasil *fine-tuning* tersebut dengan menggunakan bahasa pemrograman **Python** dan pustaka *machine learning* seperti **PyTorch** dan **Hugging Face Transformers**?
3. Bagaimana pengaruh penerapan strategi *fine-tuning* yang berbeda (*Freeze Encoder dengan melatih hanya Decoder* versus *parameter-efficient fine-tuning* dengan **LoRA**) terhadap **kinerja model Whisper Small** dalam mengenali ucapan bahasa Minangkabau, terutama dalam konteks dataset yang sangat terbatas (*extremely low-resource*) dan keterbatasan sumber daya komputasi?
4. Bagaimana cara mengukur dan mengevaluasi **performa model hasil fine-tuning** menggunakan metrik kuantitatif seperti *Word Error Rate (WER)* dan *Character Error Rate (CER)*, serta melakukan analisis kualitatif terhadap kesalahan transkripsi dengan mengkategorikan jenis kesalahan (fonetik, leksikal, dan kontekstual) pada kondisi rekaman?
5. Bagaimana hasil implementasi dan pengujian sistem ASR yang dikembangkan dapat memberikan **kontribusi nyata terhadap pelestarian bahasa daerah** dan pengembangan teknologi pengenalan suara untuk bahasa lokal di Indonesia?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah dikemukakan, penelitian ini memiliki beberapa tujuan sebagai berikut:

1. **Mengadaptasi model OpenAI Whisper Small** melalui proses *fine-tuning* agar mampu mengenali dan mentranskripsi ucapan dalam **bahasa Minangkabau** dengan tingkat akurasi yang lebih baik dibandingkan model dasar (*pretrained model*).
2. **Merancang dan mengimplementasikan sistem Automatic Speech Recognition (ASR)** berbasis hasil *fine-tuning* model Whisper menggunakan bahasa pemrograman **Python** dan pustaka *deep learning* seperti **PyTorch** serta **Hugging Face Transformers**.
3. **Menerapkan dan membandingkan strategi fine-tuning yang berbeda**, yaitu *Freeze Encoder (melatih hanya Decoder)* dan *parameter-efficient fine-tuning* dengan **LoRA**, untuk menentukan metode yang paling efektif dan efisien dalam konteks dataset yang sangat terbatas (*extremely low-resource*), dengan mempertimbangkan trade-off antara akurasi, waktu pelatihan, dan kebutuhan memori komputasi.
4. **Mengevaluasi performa model hasil fine-tuning** menggunakan metrik kuantitatif seperti *Word Error Rate (WER)* dan *Character Error Rate (CER)*, serta melakukan analisis kualitatif dengan mengkategorikan kesalahan transkripsi berdasarkan tipe kesalahan (fonetik, leksikal, dan kontekstual) untuk mengidentifikasi pola kesalahan dominan dan memberikan rekomendasi perbaikan model.
5. **Menghasilkan prototipe sistem ASR bahasa Minangkabau yang fungsional dan terverifikasi**, serta memberikan kontribusi terhadap upaya pelestarian bahasa daerah melalui penerapan teknologi kecerdasan buatan di bidang pengenalan suara multibahasa.

1.4 Batasan Masalah

Agar penelitian ini memiliki ruang lingkup yang terarah dan dapat diselesaikan secara realistis dalam waktu yang terbatas, maka ditetapkan beberapa batasan masalah sebagai berikut:

1. Penelitian ini hanya berfokus pada **implementasi *fine-tuning* model OpenAI Whisper Small** untuk pengenalan ucapan (*Automatic Speech Recognition*) pada **bahasa Minangkabau**.
2. Dataset yang digunakan dalam penelitian ini bersumber dari **LibriVox Indonesia**, sebuah korpus audio publik yang menyediakan rekaman pembacaan *Universal Declaration of Human Rights* (UDHR) dalam bahasa Minangkabau beserta transkripsi teksnya. Dataset ini merupakan dataset *extremely low-resource* dengan total **sekitar 1.3 jam audio** (155 segmen audio dengan durasi rata-rata 30 detik). Untuk memaksimalkan penggunaan data yang sangat terbatas dan memperoleh evaluasi yang lebih robust, penelitian ini menggunakan pendekatan **5-Fold Cross-Validation**, di mana setiap *fold* terdiri dari sekitar 124 audio untuk *training* dan 31 audio untuk *testing*. Dataset terbatas pada domain **pembacaan formal** (*formal reading*) dari teks deklarasi hak asasi manusia, sehingga model yang dihasilkan merupakan *prototype* pada domain spesifik dan performa pada percakapan spontan atau konteks informal mungkin berbeda. Penelitian ini tidak mencakup proses pembuatan atau pengumpulan dataset baru dalam skala besar, melainkan fokus pada pemanfaatan dan *preprocessing* data yang sudah tersedia dengan pendekatan *transfer learning* yang efisien untuk mengatasi keterbatasan data.
3. Model yang diujikan dibatasi pada **varian Whisper Small** dengan dua strategi *fine-tuning* yang sesuai untuk skenario *extremely low-resource*: (1) **Freeze Encoder** (membekukan seluruh lapisan *encoder* dan hanya melatih

decoder) dan (2) **LoRA** (*Low-Rank Adaptation*), tanpa membandingkan dengan varian Whisper lainnya (Tiny, Base, Medium, Large) atau strategi *full fine-tuning* yang memerlukan data lebih banyak.

4. Evaluasi performa model dilakukan menggunakan metrik kuantitatif **Word Error Rate (WER)** dan **Character Error Rate (CER)**. Selain itu, dilakukan analisis kualitatif terhadap kesalahan transkripsi dengan mengkategorikan jenis kesalahan ke dalam beberapa tipe, yaitu: (1) **kesalahan fonetik** (misalnya kesalahan pengenalan fonem vokal atau konsonan khas Minangkabau), (2) **kesalahan leksikal** (misalnya kesalahan pada kata serapan dari bahasa Indonesia atau bahasa asing yang diucapkan dalam bahasa Minangkabau), dan (3) **kesalahan kontekstual** (kesalahan pada kata yang bergantung pada konteks kalimat). Analisis kualitatif ini akan dilakukan secara manual terhadap sampel hasil transkripsi untuk mengidentifikasi pola kesalahan yang dominan dan memberikan rekomendasi perbaikan model. Evaluasi subjektif berbasis pengguna akhir (*user study*) tidak termasuk dalam ruang lingkup penelitian ini.
5. Implementasi sistem ASR dilakukan menggunakan bahasa pemrograman **Python** dengan pustaka **PyTorch** dan **Hugging Face Transformers**, serta dijalankan dalam lingkungan komputasi lokal atau GPU yang tersedia. Integrasi ke aplikasi lain (misalnya mobile atau web) tidak menjadi fokus utama penelitian ini.
6. Penelitian ini tidak membahas aspek keamanan data, privasi pengguna, maupun optimasi energi selama pelatihan model, melainkan berfokus pada **adaptasi model dan analisis performa hasil fine-tuning**.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik dari segi akademik maupun praktis, yaitu sebagai berikut:

1. Manfaat Akademik:

- (a) Memberikan kontribusi terhadap pengembangan ilmu pengetahuan di bidang **kecerdasan buatan (Artificial Intelligence)** dan **pemrosesan suara (Speech Processing)**, khususnya dalam konteks *Automatic Speech Recognition* (ASR) untuk bahasa dengan sumber daya terbatas.
- (b) Menjadi **referensi ilmiah** dan bahan pembelajaran bagi mahasiswa atau peneliti lain yang tertarik untuk melakukan penelitian serupa dalam bidang *speech-to-text*, *low-resource language modeling*, dan *model adaptation*.
- (c) Menunjukkan penerapan teknik *fine-tuning* pada model **OpenAI Whisper Small** menggunakan pustaka **PyTorch** dan **Hugging Face Transformers**, sehingga dapat menjadi dasar penelitian lanjutan terkait efisiensi model, optimasi pelatihan, dan adaptasi multibahasa.

2. Manfaat Praktis:

- (a) Menghasilkan **prototipe sistem pengenalan suara bahasa Minangkabau** yang dapat digunakan sebagai alat bantu transkripsi otomatis untuk **konten audio formal**, seperti pembacaan teks resmi, dokumentasi narasi budaya, atau rekaman arsip berbahasa Minangkabau yang bersifat formal. Perlu dicatat bahwa performa model pada percakapan spontan atau konteks informal mungkin memerlukan adaptasi lebih lanjut dengan data domain yang sesuai.
- (b) Mendukung **pelestarian bahasa daerah** melalui pemanfaatan teknologi AI yang dapat mempermudah **digitalisasi dan dokumentasi** arsip audio berbahasa Minangkabau, terutama untuk rekaman-rekaman historis atau konten edukatif yang bersifat formal.
- (c) Memberikan **fondasi penelitian dan contoh implementasi** untuk pengembangan sistem ASR bahasa daerah Indonesia, yang dapat menjadi *baseline* atau titik awal bagi penelitian lanjutan dengan dataset yang lebih beragam (termasuk percakapan spontan, podcast,

atau konteks informal) untuk mencapai sistem ASR bahasa Minangkabau yang lebih general-purpose.

1.6 Sistematika Penulisan

Sistematika penulisan berisi penjelasan mengenai susunan bab dalam laporan tugas akhir ini. Setiap bab disusun secara sistematis agar pembaca dapat memahami alur penelitian yang dilakukan, mulai dari tahap perumusan masalah hingga kesimpulan akhir. Adapun sistematika penulisan tugas akhir ini adalah sebagai berikut:

Bab I: Pendahuluan

Bab ini berisi penjelasan mengenai **latar belakang penelitian, rumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian**, serta **sistematika penulisan**. Bab ini memberikan gambaran umum mengenai alasan dan arah penelitian yang dilakukan, sehingga pembaca memahami konteks dan ruang lingkup penelitian ini.

Bab II: Tinjauan Pustaka

Bab ini membahas **landasan teori** dan **penelitian terdahulu** yang relevan dengan topik tugas akhir, meliputi konsep dasar *Automatic Speech Recognition (ASR)*, model **OpenAI Whisper**, teknik *fine-tuning* dan *parameter-efficient fine-tuning* seperti **LoRA**, serta pembahasan mengenai **bahasa Minangkabau sebagai bahasa low-resource**. Bab ini menjadi dasar konseptual dan teoritis bagi metode yang digunakan dalam penelitian.

Bab III: Metodologi Penelitian

Bab ini menjelaskan **tahapan penelitian** yang dilakukan, termasuk rancangan sistem, arsitektur model, dataset yang digunakan, tahapan *fine-tuning*, lingkungan pengujian, serta metode evaluasi performa. Selain itu, bab ini juga menjelaskan alur kerja penelitian mulai dari pengumpulan data,

pelatihan model, hingga proses evaluasi hasil.

Bab IV: Implementasi dan Hasil Penelitian

Bab ini membahas **implementasi model dan sistem** yang dikembangkan, hasil pengujian terhadap model *Whisper Small* yang telah di-*fine-tune*, serta analisis performa berdasarkan metrik *Word Error Rate (WER)*, *Character Error Rate (CER)*, dan evaluasi kualitatif terhadap hasil transkripsi. Pada bagian ini juga dijelaskan perbandingan antara model dasar dan model hasil adaptasi terhadap bahasa Minangkabau.

Bab V: Kesimpulan dan Saran

Bab ini berisi **kesimpulan** yang diperoleh dari hasil penelitian dan **saran** untuk pengembangan lebih lanjut. Kesimpulan mencakup hasil utama penelitian, tingkat keberhasilan implementasi sistem ASR bahasa Minangkabau, serta potensi pengembangan model atau sistem di masa depan.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Pustaka

Bagian ini mengulas penelitian-penelitian terdahulu yang relevan dengan topik *fine-tuning* Whisper dan *Automatic Speech Recognition* (ASR) untuk bahasa *low-resource*. Setiap penelitian dianalisis secara kritis untuk mengidentifikasi kontribusi utama, keterbatasan, serta relevansinya dengan penelitian ini yang berfokus pada adaptasi Whisper Small untuk bahasa Minangkabau.

Tabel 2.1 Ringkasan Penelitian Terdahulu (2022–2025)

No.	Penelitian & Penulis	Fokus Masalah	Pendekatan & Kontribusi	Temuan Utama & Relevansi
1	<i>Robust Speech Recognition via Large-Scale Weak Supervision</i> Radford et al. [1] (2023)	Membangun ASR multibahasa yang <i>robust</i> dengan kemampuan generalisasi <i>zero-shot</i> tanpa <i>fine-tuning</i> spesifik	Pelatihan Whisper pada ~680k jam data audio multibahasa dengan paradigma <i>weak supervision</i> ; arsitektur <i>Transformer encoder-decoder</i>	Kinerja kuat di berbagai <i>benchmark</i> ASR; fondasi untuk adaptasi bahasa <i>low-resource</i> ; keterbatasan pada bahasa <i>under-represented</i> memotivasi <i>fine-tuning</i> untuk Minangkabau

No.	Penelitian & Penulis	Fokus Masalah	Pendekatan & Kontribusi	Temuan Utama & Relevansi
2	<i>Exploration of Whisper Fine-Tuning Strategies for Low-Resource ASR</i> Liu, Yang, & Qu [2] (2024)	Identifikasi strategi <i>fine-tuning</i> Whisper yang paling efektif untuk bahasa <i>low-resource</i>	Studi komparatif <i>full fine-tuning</i> , <i>partial fine-tuning</i> , <i>adapters</i> , dan PEFT (LoRA) pada berbagai bahasa <i>low-resource</i>	Semua strategi menurunkan WER; <i>trade-off</i> jelas antara akurasi dan efisiensi; panduan praktis untuk pemilihan strategi; basis untuk perbandingan <i>Freeze Encoder</i> vs LoRA pada Minangkabau
3	<i>Fine-tuning Whisper on Low-Resource Languages for Real-World Applications</i> Timmel et al. [3] (2024)	Model kehilangan kemampuan segmentasi dan <i>timestamping</i> saat dilatih hanya pada kalimat pendek	Rekonstruksi korpus <i>long-form</i> sintetis dari data kalimat pendek; evaluasi pada Swiss German	SOTA untuk Swiss German; segmentasi dan <i>timestamping</i> tetap terjaga; relevan untuk Minangkabau jika data <i>long-form</i> terbatas

No.	Penelitian & Penulis	Fokus Masalah	Pendekatan & Kontribusi	Temuan Utama & Relevansi
4	<i>Towards Efficiently Whisper Fine-tuning with Monotonic Alignments</i> Zhuang et al. [7] (2025)	<i>Fine-tuning</i> standar tidak mengoptimalkan keselarasan monotonic audio-teks secara eksplisit	Penambahan <i>soft monotonic alignment loss</i> tanpa modifikasi arsitektur model	WER turun konsisten pada ASR multibahasa dan <i>speech translation</i> ; overhead minimal; opsi pengembangan potensial untuk Minangkabau
5	<i>Indonesian Automatic Speech Recognition with XLSR-53</i> Arisaputra & Zahra [8] (2022)	Mencapai akurasi ASR kompetitif untuk bahasa Indonesia dengan data terbatas	<i>Fine-tuning</i> XLSR-53 (~24 jam) dikombinasikan dengan <i>language model</i> 4-gram (KenLM) pada <i>decoding</i>	WER turun drastis dari ~20% ke ~12% dengan LM; bukti <i>transfer learning</i> multibahasa efektif; mendukung hipotesis untuk bahasa daerah seperti Minangkabau

No.	Penelitian & Penulis	Fokus Masalah	Pendekatan & Kontribusi	Temuan Utama & Relevansi
6	<i>Breaking the Transcription Bottleneck: ASR for Extremely Low-Resource Languages</i> Liang & Levow [9] (2025)	Transkripsi bahasa dengan data sangat minimal (<1 jam) dalam konteks linguistik lapangan	<i>Benchmark</i> MMS vs. XLS-R pada 5 bahasa sangat <i>low-resource</i> dengan berbagai skala data	MMS unggul pada <1 jam data; paritas dengan XLS-R saat data bertambah; memberikan konteks batas bawah data untuk <i>fine-tuning</i> efektif

2.1.1 Model Whisper dan Fondasi Teoritis

Radford et al. [1] memperkenalkan Whisper, model ASR multibahasa yang dilatih pada sekitar 680,000 jam data audio dengan paradigma *weak supervision*. Penelitian ini menjawab tantangan fundamental dalam membangun sistem ASR yang *robust* dan mampu melakukan generalisasi *zero-shot* pada berbagai bahasa dan kondisi audio tanpa memerlukan *fine-tuning* pada dataset spesifik. Whisper menggunakan arsitektur *Transformer encoder-decoder* yang dilatih pada tugas-tugas multibahasa, termasuk transkripsi dan translasi ucapan. Evaluasi komprehensif menunjukkan bahwa Whisper mencapai kinerja yang kuat di berbagai *benchmark* ASR dan translasi wicara dalam skenario *zero-shot*, membuktikan kemampuan generalisasi lintas bahasa yang luar biasa.

Meskipun Whisper menyediakan fondasi yang kokoh untuk adaptasi ke bahasa lokal, penelitian ini juga mengidentifikasi keterbatasan penting: bahasa dan dialek yang kurang terwakili dalam data pelatihan (*under-represented languages*) masih menghasilkan tingkat kesalahan yang relatif tinggi. Temuan ini memotivasi kebutuhan untuk *fine-tuning* model pada bahasa-bahasa *low-*

resource seperti Minangkabau. Dalam konteks penelitian ini, Whisper menjadi model dasar yang akan diadaptasi melalui *fine-tuning* dengan strategi yang efisien untuk meningkatkan akurasi pada bahasa target.

2.1.2 Strategi Fine-Tuning untuk Bahasa Low-Resource

Liu, Yang, dan Qu [2] melakukan studi komparatif komprehensif terhadap berbagai strategi *fine-tuning* Whisper pada skenario data terbatas. Penelitian ini mengeksplorasi pendekatan yang beragam, termasuk *full fine-tuning* (melatih seluruh parameter model), *partial fine-tuning* dengan *freezing* sebagian lapisan, *re-initialization* lapisan atas, penggunaan *adapters*, serta metode Parameter-Efficient Fine-Tuning (PEFT) seperti LoRA. Evaluasi dilakukan pada berbagai bahasa *low-resource* untuk mengidentifikasi pola umum yang dapat dijadikan panduan praktis.

Hasil penelitian menunjukkan bahwa semua strategi yang diuji mampu menurunkan *Word Error Rate* (WER) dibandingkan model dasar, namun terdapat *trade-off* yang jelas antara akurasi dan efisiensi komputasi. *Full fine-tuning* cenderung menghasilkan akurasi tertinggi tetapi memerlukan memori GPU yang besar dan waktu pelatihan yang lama, sementara metode PEFT seperti LoRA menawarkan efisiensi signifikan dengan penurunan akurasi yang minimal. Keunggulan penelitian ini terletak pada panduan praktis untuk memilih strategi berdasarkan keterbatasan data dan sumber daya komputasi yang tersedia. Namun, keterbatasannya adalah fokus yang luas tanpa analisis mendalam pada satu bahasa atau dialek spesifik secara longitudinal.

Penelitian ini memberikan kontribusi penting sebagai dasar pemilihan strategi untuk bahasa Minangkabau. Dalam skripsi ini, perbandingan langsung antara *Freeze Encoder* dan LoRA pada Whisper Small akan dilakukan untuk mengidentifikasi strategi yang paling sesuai dalam konteks bahasa Minangkabau, dengan mempertimbangkan keterbatasan data dan infrastruktur komputasi yang tersedia.

2.1.3 Adaptasi untuk Aplikasi Dunia Nyata

Timmel et al. [3] mengangkat isu praktis penting dalam *fine-tuning* Whisper untuk bahasa *low-resource*, khususnya Swiss German. Penelitian ini mengidentifikasi bahwa kekurangan korpus audio *long-form* (rekaman panjang kontinu) menyebabkan model kehilangan kemampuan segmentasi dan *timestamping* ketika hanya dilatih pada kalimat-kalimat pendek yang terisolasi. Untuk mengatasi masalah ini, peneliti mengembangkan metode untuk menyusun ulang data kalimat pendek menjadi korpus audio *long-form* sintetis, kemudian melakukan *fine-tuning* Whisper pada data yang telah direkonstruksi tersebut.

Hasil evaluasi menunjukkan peningkatan signifikan pada aplikasi dunia nyata, terutama dalam menangani percakapan panjang dan kontinu, sementara kemampuan segmentasi dan *timestamping* tetap terjaga dengan baik. Pendekatan ini mencapai *state-of-the-art* untuk Swiss German dan memberikan solusi praktis ketika data *long-form* asli sangat terbatas. Namun, validasi lintas bahasa daerah lain masih diperlukan untuk memastikan generalisasi metode ini.

Relevansi penelitian ini terhadap bahasa Minangkabau sangat tinggi, terutama jika dataset yang tersedia dari LibriVox Indonesia memiliki keterbatasan dalam bentuk *long-form*. Strategi penyusunan korpus *long-form* sintetis dapat diadopsi sebagai bagian dari *preprocessing* data untuk meningkatkan kemampuan model dalam menangani konteks yang lebih panjang dan kompleks.

2.1.4 Optimisasi Proses Fine-Tuning

Zhuang et al. [7] mengusulkan pendekatan inovatif untuk meningkatkan efisiensi *fine-tuning* Whisper dengan menambahkan *soft monotonic alignment loss* selama proses pelatihan. Penelitian ini didasarkan pada observasi bahwa *fine-tuning* standar tidak secara eksplisit mengoptimalkan keselarasan monotonic antara sekuens audio dan teks, yang seharusnya merupakan karakteristik alamiah dalam transkripsi ucapan. Dengan menambahkan *loss function* tambahan yang mendorong alignment monotonic tanpa mengubah arsitektur model, peneliti

mencapai penurunan WER yang konsisten pada berbagai tugas ASR multibahasa, serta perbaikan pada tugas *speech translation*.

Keunggulan pendekatan ini adalah peningkatan akurasi dengan *computational overhead* yang minimal, karena tidak memerlukan modifikasi arsitektur atau penambahan parameter yang signifikan. Namun, metode ini memerlukan penyetelan *weight* untuk *loss function* tambahan, dan dampaknya pada bahasa yang sangat *low-resource* (dengan data sangat terbatas) masih memerlukan studi lanjutan. Dalam konteks penelitian ini, metode *monotonic alignment* menjadi opsi pengembangan potensial jika performa *baseline fine-tuning* untuk bahasa Minangkabau belum mencapai tingkat yang memadai.

2.1.5 Transfer Learning Multibahasa

Arisaputra dan Zahra [8] mendemonstrasikan efektivitas *transfer learning* untuk ASR bahasa Indonesia dengan data terbatas menggunakan model XLSR-53. Penelitian ini melakukan *fine-tuning* XLSR-53 pada sekitar 24 jam data audio bahasa Indonesia, dikombinasikan dengan *language model* berbasis KenLM 4-gram pada tahap *decoding* untuk meningkatkan akurasi transkrip. Hasil eksperimen menunjukkan penurunan WER yang dramatis dari sekitar 20% menjadi 12% dengan penambahan *language model*, membuktikan bahwa pendekatan *transfer learning* dari model *pre-trained* multibahasa sangat efektif untuk bahasa Indonesia.

Meskipun penelitian ini berfokus pada bahasa Indonesia baku dan tidak membahas dialek atau bahasa daerah, temuan ini memberikan bukti kuat bahwa representasi multibahasa yang dipelajari oleh model seperti XLSR dapat ditransfer ke bahasa Indonesia dengan efektif. Sebagai perbandingan non-Whisper, penelitian ini mendukung hipotesis bahwa model *pre-trained* multibahasa yang dikombinasikan dengan *fine-tuning* merupakan pendekatan yang viabel untuk bahasa Indonesia dan bahasa daerah seperti Minangkabau.

2.1.6 ASR untuk Bahasa dengan Data Sangat Terbatas

Liang dan Levow [9] mengeksplorasi tantangan transkripsi untuk bahasa-bahasa yang memiliki data sangat minimal, bahkan kurang dari 1 jam audio, dalam konteks penelitian linguistik lapangan (*fieldwork linguistics*). Penelitian ini membandingkan performa model MMS (*Massively Multilingual Speech*) dan XLS-R pada 5 bahasa yang sangat *low-resource* dengan berbagai skala data. Hasil evaluasi menunjukkan bahwa MMS unggul ketika data pelatihan sangat terbatas (kurang dari 1 jam), sementara performa kedua model mencapai paritas ketika data bertambah (lebih dari 1 jam).

Temuan ini memberikan panduan praktis untuk pemilihan arsitektur model berdasarkan ketersediaan data, meskipun fokus penelitian ini adalah pada MMS dan XLS-R, bukan Whisper. Meskipun demikian, penelitian ini memberikan konteks penting tentang batas bawah data yang diperlukan untuk *fine-tuning* yang efektif. Dalam penelitian ini, meskipun dataset Minangkabau dari LibriVox Indonesia relatif terbatas, fokus tetap pada Whisper Small yang telah terbukti memiliki kemampuan *transfer learning* yang kuat, dengan eksplorasi strategi *fine-tuning* yang efisien melalui perbandingan *Freeze Encoder* dan LoRA.

2.1.7 Sintesis dan Posisi Penelitian

Tinjauan terhadap literatur terbaru mengungkapkan beberapa temuan penting yang menjadi landasan penelitian ini. Pertama, pelatihan skala besar dengan paradigma *weak supervision* seperti yang dilakukan pada Whisper [1] memberikan fondasi representasi multibahasa yang kuat untuk adaptasi bahasa *low-resource*. Kedua, pemilihan strategi *fine-tuning* harus mempertimbangkan *trade-off* antara akurasi dan efisiensi komputasi [2], di mana metode seperti LoRA menawarkan alternatif yang efisien dibandingkan melatih lebih banyak parameter. Ketiga, rekayasa korpus *long-form* dapat membantu mempertahankan kemampuan segmentasi dan *timestamping* untuk aplikasi dunia nyata [3]. Keempat, modifikasi fungsi objektif seperti *monotonic alignment loss* dapat

meningkatkan akurasi tanpa mengubah arsitektur [7]. Kelima, pembelajaran multibahasa dapat ditransfer secara efektif ke bahasa lokal dan daerah [8].

Berdasarkan temuan-temuan tersebut, penelitian ini mengidentifikasi beberapa peluang pengembangan. Pertama, perbandingan sistematis antara *Freeze Encoder* dan LoRA pada Whisper Small untuk bahasa Minangkabau akan memberikan *insights* praktis tentang efisiensi penggunaan memori GPU tanpa mengorbankan akurasi secara signifikan. Kedua, jika data *long-form* Minangkabau terbatas, penyusunan korpus *long-form* sintetis dapat diadopsi untuk meningkatkan kemampuan model dalam konteks yang lebih panjang. Ketiga, eksplorasi *alignment-based loss* dan/atau integrasi *language model* pada tahap *decoding* dapat menjadi strategi lanjutan untuk penurunan WER lebih lanjut.

2.2 Dasar Teori

Bagian ini menjelaskan landasan teoritis yang mendukung metodologi penelitian pada Bab III. Pembahasan disusun secara hierarkis dari konsep umum hingga spesifik: (i) *Automatic Speech Recognition* (ASR) dan tantangan bahasa *low-resource*, (ii) *transfer learning* dan model *pre-trained*, (iii) arsitektur Whisper, (iv) strategi *fine-tuning* (*Full* dan LoRA), (v) metrik evaluasi ASR, serta (vi) *preprocessing* data audio dan teks.

2.2.1 Automatic Speech Recognition (ASR)

Automatic Speech Recognition (ASR) adalah teknologi yang mengkonversi sinyal audio ucapan manusia menjadi teks tertulis secara otomatis [1]. Sistem ASR modern menggunakan pendekatan *deep learning* yang memanfaatkan arsitektur jaringan saraf untuk mempelajari pola kompleks dalam data audio. Perkembangan ASR telah mengalami evolusi signifikan, dari sistem berbasis *Hidden Markov Model* (HMM) hingga arsitektur *end-to-end* berbasis *Transformer* [10] yang mampu memodelkan hubungan jarak jauh dalam sekuens

audio.

Tantangan utama dalam ASR adalah variabilitas tinggi dalam ucapan manusia yang dipengaruhi oleh faktor-faktor seperti aksen, dialek, kecepatan bicara, *background noise*, dan karakteristik *speaker* individual. Untuk bahasa dengan sumber daya tinggi seperti bahasa Inggris, ketersediaan dataset besar (ribuan jam audio teranotasi) memungkinkan model mencapai akurasi tinggi. Namun, untuk bahasa daerah *low-resource* seperti Minangkabau, minimnya dataset merupakan hambatan signifikan [2].

2.2.1.1 Bahasa Low-Resource dan Tantangannya

Bahasa *low-resource* didefinisikan sebagai bahasa dengan keterbatasan data teranotasi yang tersedia untuk pelatihan model ASR [9]. Tantangan spesifik bahasa *low-resource* meliputi: (i) kurangnya korpus audio-transkrip berkualitas tinggi, (ii) minimnya keberagaman *speaker* dan konteks penggunaan dalam dataset, (iii) keterbatasan sumber daya komputasi untuk pengumpulan data skala besar, dan (iv) variasi dialek yang tidak tercakup dalam data yang tersedia [3], [11].

Penelitian terbaru menunjukkan bahwa pendekatan *transfer learning* dari model *pre-trained* multibahasa dapat mengatasi keterbatasan data pada bahasa *low-resource* [8], [12]. Model yang telah dilatih pada bahasa-bahasa dengan sumber daya tinggi dapat di-adaptasi ke bahasa target dengan jumlah data yang relatif sedikit, memanfaatkan representasi linguistik dan akustik yang telah dipelajari dari bahasa-bahasa lain.

2.2.2 Transfer Learning dan Model Pre-Trained

Transfer learning adalah paradigma *machine learning* di mana pengetahuan yang dipelajari dari satu domain (*source domain*) ditransfer ke domain lain (*target domain*) untuk meningkatkan performa dengan data pelatihan yang terbatas [13]. Dalam konteks ASR, *transfer learning* memungkinkan model yang telah dilatih

pada bahasa-bahasa dengan sumber daya tinggi untuk diadaptasi ke bahasa *low-resource* melalui proses *fine-tuning*.

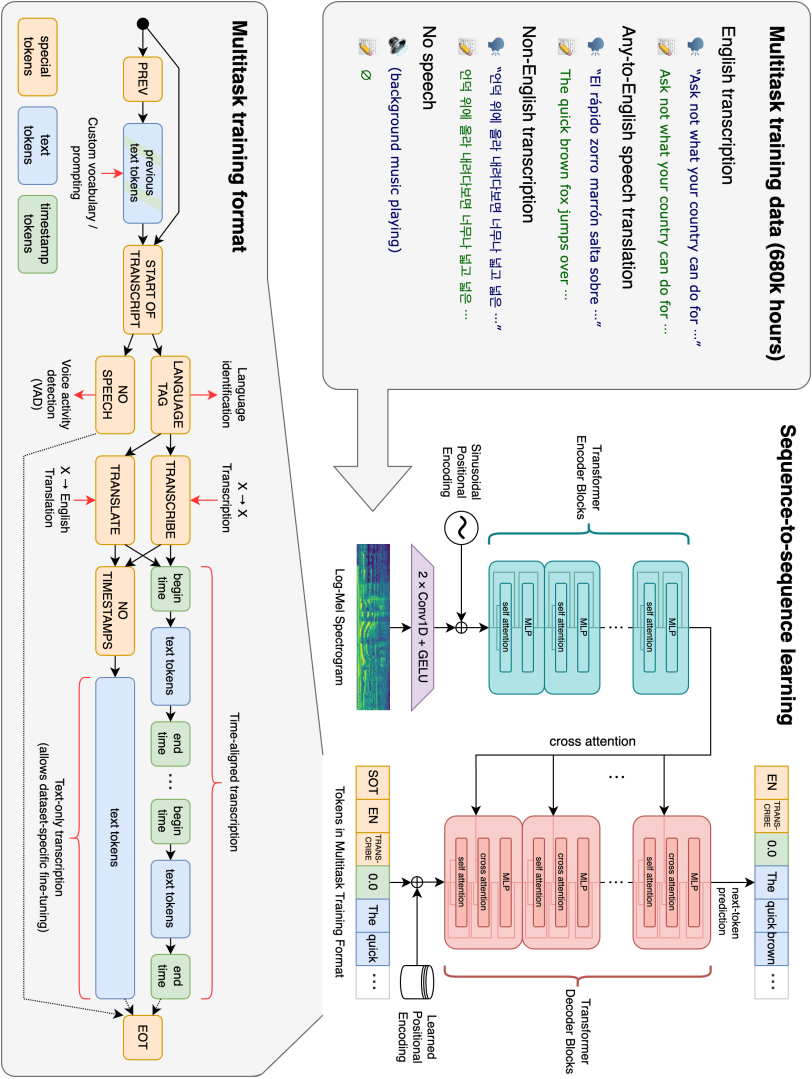
2.2.2.1 Model Pre-Trained untuk ASR

Model *pre-trained* multibahasa seperti wav2vec 2.0 [14], XLS-R [15], HuBERT [16], WavLM [17], dan Whisper [1] telah menunjukkan kemampuan luar biasa dalam *transfer learning* untuk ASR. Model-model ini dilatih pada ratusan ribu jam data audio multibahasa, memungkinkan mereka mempelajari representasi akustik dan linguistik yang generik dan dapat ditransfer ke bahasa-bahasa baru.

2.2.3 Whisper dan Arsitekturnya

Whisper adalah model *general-purpose speech recognition* yang dilatih pada dataset audio berskala besar yang beragam. Berbeda dengan model ASR konvensional yang umumnya dilatih pada dataset terkurasi, Whisper dilatih menggunakan pendekatan *weakly supervised* pada 680.000 jam data audio multibahasa dan multitugas yang dikumpulkan dari internet [1]. Skala data yang masif dan keragaman sumber data ini memberikan Whisper kemampuan *robustness* yang tinggi terhadap berbagai aksen, kebisingan latar belakang, dan bahasa teknis, serta memungkinkannya melakukan transkripsi dalam berbagai bahasa maupun terjemahan ke dalam bahasa Inggris.

Secara arsitektural, Whisper menggunakan model *Transformer* [10] dalam konfigurasi *encoder-decoder* yang telah terbukti efektif untuk tugas sekuens-ke-sekuens. Arsitektur ini terdiri dari dua komponen utama: *encoder* yang memproses representasi audio dan *decoder* yang menghasilkan transkrip teks.



Gambar 2.1 Arsitektur dan Multitask Training Format Whisper

2.2.3.1 Varian Model dan Parameter

Whisper tersedia dalam berbagai ukuran model yang menawarkan keseimbangan berbeda antara akurasi dan kebutuhan sumber daya komputasi.

OpenAI merilis lima ukuran model utama: *Tiny*, *Base*, *Small*, *Medium*, dan *Large*, serta varian *Turbo* yang dioptimalkan untuk kecepatan. Pemilihan ukuran model sangat bergantung pada ketersediaan memori GPU (VRAM) dan kebutuhan kecepatan inferensi.

Tabel 2.2 menyajikan perbandingan spesifikasi teknis untuk setiap varian model Whisper. Model *English-only* (berakhiran *.en*) dilatih khusus pada data bahasa Inggris, sedangkan model *Multilingual* dilatih pada data multibahasa termasuk bahasa Indonesia.

Tabel 2.2 Spesifikasi dan Kebutuhan Sumber Daya Varian Model Whisper

Size	Parameters	English-only	Multilingual	VRAM	Relative Speed
Tiny	39 M	tiny.en	tiny	~1 GB	~10x
Base	74 M	base.en	base	~1 GB	~7x
Small	244 M	small.en	small	~2 GB	~4x
Medium	769 M	medium.en	medium	~5 GB	~2x
Large	1550 M	N/A	large	~10 GB	1x
Turbo	809 M	N/A	turbo	~6 GB	~8x

Dalam penelitian ini, model **Whisper Small** dipilih sebagai basis untuk *fine-tuning*. Dengan 244 juta parameter, Whisper Small menawarkan keseimbangan optimal: cukup kompleks untuk menangkap nuansa bahasa Minangkabau, namun tetap efisien untuk dilatih pada GPU dengan VRAM terbatas dibandingkan model Medium atau Large yang membutuhkan sumber daya jauh lebih besar.

2.2.3.2 Encoder

Encoder Whisper menerima input berupa representasi *log-Mel spectrogram* dari audio dengan 80 dimensi frekuensi. Spektrogram dihitung dengan *window size* 25ms dan *hop length* 10ms, menghasilkan 100 frame per detik audio. Input ini diproses melalui beberapa lapisan *Transformer encoder* yang menggunakan mekanisme *multi-head self-attention* [18] untuk menangkap dependensi temporal jarak jauh dalam sekuens audio.

Setiap lapisan *encoder* terdiri dari dua sublapisan utama: (i) *multi-head self-attention* yang memungkinkan model memperhatikan bagian-bagian berbeda dari input audio secara paralel, dan (ii) *feed-forward network* (FFN) yang menerapkan transformasi non-linear pada representasi. Kedua sublapisan ini dilengkapi dengan *residual connections* dan *layer normalization* [19] untuk stabilitas pelatihan.

2.2.3.3 Decoder

Decoder Whisper adalah *autoregressive Transformer decoder* yang menghasilkan transkrip teks secara sekuensial, satu *token* pada satu waktu. *Decoder* menggunakan *masked self-attention* untuk mencegah model melihat *token* masa depan selama pelatihan, serta *cross-attention* untuk memperhatikan representasi audio dari *encoder*.

Proses generasi teks dimulai dengan *token* khusus yang mengindikasikan bahasa target dan tugas yang akan dilakukan (*transcription* atau *translation*). *Decoder* kemudian secara berulang memprediksi *token* berikutnya berdasarkan *token-token* sebelumnya dan representasi audio, hingga *token* akhir sekuens diprediksi.

2.2.3.4 Whisper Small

Whisper tersedia dalam berbagai ukuran model, dari *Tiny* (39M parameter) hingga *Large* (1550M parameter). Whisper Small, dengan 244 juta parameter, menawarkan keseimbangan optimal antara performa dan efisiensi komputasi [2]. Arsitektur Whisper Small terdiri dari 12 lapisan *encoder* dan 12 lapisan *decoder*, dengan dimensi *hidden state* 768 dan 12 *attention heads* per lapisan. Ukuran *vocabulary* adalah 51,864 *tokens* yang mencakup representasi karakter dan *subword* untuk berbagai bahasa.

2.2.4 Fine-Tuning: Konsep dan Strategi

Fine-tuning adalah proses adaptasi model *pre-trained* ke tugas atau domain spesifik dengan melanjutkan pelatihan pada dataset target yang lebih kecil [13]. Dalam konteks ASR untuk bahasa *low-resource*, *fine-tuning* memungkinkan model yang telah dilatih pada bahasa-bahasa lain untuk disesuaikan dengan karakteristik linguistik dan akustik bahasa target.

2.2.4.1 Full Fine-Tuning

Full fine-tuning adalah pendekatan tradisional di mana seluruh parameter model (atau sebagian besar parameter, kecuali lapisan ekstraksi fitur yang sering di-*freeze*) di-*update* selama proses pelatihan pada dataset target [2]. Pendekatan ini memberikan fleksibilitas maksimal bagi model untuk beradaptasi dengan bahasa target, tetapi memerlukan sumber daya komputasi yang signifikan karena gradien harus dihitung dan parameter harus di-*update* untuk seluruh jaringan.

Keuntungan *full fine-tuning* adalah potensi akurasi maksimal karena semua parameter dapat disesuaikan dengan data target. Namun, keterbatasannya termasuk: (i) kebutuhan memori GPU yang besar untuk menyimpan gradien dan *optimizer states* untuk semua parameter, (ii) waktu pelatihan yang lebih lama, dan (iii) risiko *overfitting* pada dataset kecil jika tidak diterapkan regularisasi yang memadai.

2.2.4.2 Freeze Encoder Fine-Tuning

Freeze Encoder Fine-Tuning adalah strategi *fine-tuning* yang membekukan (*freeze*) seluruh parameter *encoder* dan hanya melatih parameter *decoder* dalam arsitektur *encoder-decoder* seperti Whisper [2]. Pendekatan ini didasarkan pada hipotesis bahwa representasi akustik yang telah dipelajari *encoder* dari data *pre-training* multibahasa bersifat universal dan dapat digunakan langsung untuk bahasa target, sementara *decoder* perlu diadaptasi untuk mempelajari pola linguistik spesifik bahasa target.

Dalam konteks Whisper Small, strategi ini membekukan 12 lapisan *encoder* Transformer (~122M parameter) dan hanya melatih 12 lapisan *decoder* Transformer (~122M parameter), sehingga mengurangi jumlah parameter yang di-*update* menjadi sekitar 50% dari total parameter model. Pendekatan ini menawarkan beberapa keuntungan untuk skenario *extremely low-resource*:

- **Mengurangi Risiko Overfitting:** Dengan hanya melatih *decoder*, model mempertahankan representasi akustik yang telah dipelajari dari 680,000 jam data *pre-training*, mengurangi risiko *overfitting* pada dataset kecil
- **Efisiensi Komputasi:** Kebutuhan memori GPU dan waktu pelatihan berkurang signifikan dibandingkan *full fine-tuning* karena gradien hanya dihitung untuk separuh parameter
- **Transfer Learning Optimal:** Memanfaatkan representasi akustik universal dari *encoder* sambil mengadaptasi komponen linguistik (*decoder*) untuk bahasa target
- **Stabilitas Training:** Dengan *encoder* yang tetap stabil, pelatihan cenderung lebih stabil dan konvergen lebih cepat

Keterbatasan strategi ini adalah model hanya dapat beradaptasi pada level linguistik melalui *decoder*, sehingga jika bahasa target memiliki karakteristik akustik yang sangat berbeda dari bahasa-bahasa dalam data *pre-training*, performa mungkin tidak optimal. Namun, untuk bahasa-bahasa yang masih memiliki kemiripan akustik dengan bahasa-bahasa dalam dataset *pre-training* Whisper (seperti bahasa Minangkabau yang berkaitan dengan bahasa Indonesia), strategi ini terbukti efektif [2].

2.2.4.3 Parameter-Efficient Fine-Tuning (PEFT)

Parameter-Efficient Fine-Tuning (PEFT) adalah keluarga metode yang bertujuan mengurangi jumlah parameter yang dilatih selama *fine-tuning* sambil mempertahankan performa yang sebanding dengan *full fine-tuning* [20]. PEFT sangat relevan untuk skenario *low-resource* di mana sumber daya komputasi

terbatas.

Metode PEFT bekerja dengan membekukan (*freeze*) sebagian besar atau seluruh parameter model *pre-trained*, dan hanya melatih sebagian kecil parameter tambahan yang diinjeksikan ke dalam arsitektur. Pendekatan ini secara drastis mengurangi kebutuhan memori dan waktu pelatihan sambil tetap memungkinkan adaptasi yang efektif ke tugas target.

2.2.4.4 LoRA (Low-Rank Adaptation)

LoRA (*Low-Rank Adaptation*) [20] adalah teknik PEFT yang sangat efisien untuk *fine-tuning* model *Transformer* besar. LoRA didasarkan pada hipotesis bahwa perubahan parameter (*weight updates*) selama *fine-tuning* memiliki dimensi intrinsik yang rendah. Artinya, adaptasi model ke tugas baru dapat diaproksimasi dengan baik menggunakan matriks *low-rank*.

Secara teknis, untuk setiap matriks *weight* $W \in \mathbb{R}^{d \times k}$ dalam lapisan model (misalnya, proyeksi *query*, *key*, atau *value* dalam *attention*), LoRA menambahkan matriks *low-rank* $\Delta W = BA$, di mana $B \in \mathbb{R}^{d \times r}$ dan $A \in \mathbb{R}^{r \times k}$, dengan $r \ll \min(d, k)$. Selama *inference*, output dihitung sebagai:

$$h = Wx + \Delta Wx = Wx + BAx \quad (\text{Rumus 2.1})$$

Parameter asli W tetap *frozen*, dan hanya matriks A dan B yang dilatih. Jumlah parameter *trainable* per lapisan adalah $r \times (d + k)$, yang jauh lebih kecil dari $d \times k$ parameter asli. Untuk Whisper Small dengan $r = 8$, LoRA hanya melatih $\sim 0.24\%$ dari total parameter model [4], menghasilkan penghematan memori dan waktu pelatihan yang signifikan.

LoRA juga memiliki keunggulan praktis: (i) dapat dengan mudah di-*merge* dengan model dasar untuk *inference* tanpa overhead latensi, (ii) memungkinkan

multi-task learning dengan menyimpan beberapa set matriks LoRA untuk tugas berbeda, dan (iii) mengurangi risiko *overfitting* karena parameter yang dilatih jauh lebih sedikit.

2.2.5 Metrik Evaluasi ASR

Evaluasi performa model ASR dilakukan menggunakan metrik yang mengukur akurasi transkrip yang dihasilkan dibandingkan dengan transkrip referensi. Dua metrik utama yang digunakan adalah *Word Error Rate* (WER) dan *Character Error Rate* (CER).

2.2.5.1 Word Error Rate (WER)

Word Error Rate (WER) adalah metrik standar untuk evaluasi ASR yang mengukur *edit distance* pada level kata antara transkrip hipotesis (textithypothesis, output model) dan transkrip referensi (*reference, ground truth*) [1]. WER dihitung sebagai:

$$\text{WER} = \frac{S + D + I}{N} \times 100\% \quad (\text{Rumus 2.2})$$

di mana S adalah jumlah substitusi (kata yang salah diganti), D adalah jumlah penghapusan (kata yang hilang dalam hipotesis), I adalah jumlah penyisipan (kata tambahan dalam hipotesis), dan N adalah total kata dalam referensi. WER dinyatakan dalam persentase, di mana nilai lebih rendah menunjukkan akurasi lebih tinggi (0% berarti sempurna, 100% atau lebih berarti kinerja sangat buruk).

2.2.5.2 Character Error Rate (CER)

Character Error Rate (CER) mengukur *edit distance* pada level karakter, dihitung dengan formula yang sama dengan WER tetapi operasi substitusi,

penghapusan, dan penyisipan diterapkan pada karakter individual. CER lebih sensitif terhadap kesalahan fonetik dan memberikan perspektif komplementer terhadap WER, terutama untuk bahasa dengan sistem penulisan yang kompleks atau ketika kesalahan parsial dalam kata perlu dipertimbangkan.

Kedua metrik ini penting karena memberikan informasi yang berbeda: WER mengukur akurasi semantik pada level kata (lebih penting untuk pemahaman makna), sementara CER memberikan granularitas lebih tinggi dan berguna untuk mengidentifikasi pola kesalahan fonetik spesifik.

2.2.6 Data Preprocessing untuk ASR

Preprocessing data adalah tahap kritis dalam pengembangan sistem ASR yang memastikan data input konsisten, berkualitas tinggi, dan kompatibel dengan persyaratan model [21].

2.2.6.1 Audio Preprocessing

Audio *preprocessing* mencakup beberapa operasi penting:

1. **Resampling:** Konversi *sampling rate* audio ke frekuensi standar yang digunakan oleh model (16 kHz untuk Whisper). Resampling memastikan konsistensi temporal audio dan kompatibilitas dengan arsitektur model.
2. **Normalisasi Amplitudo:** Standarisasi volume audio di seluruh dataset untuk menghindari bias model terhadap audio dengan volume tertentu. Normalisasi dapat dilakukan dengan teknik seperti *peak normalization* atau *RMS normalization*.
3. **Segmentasi:** Pemotongan audio panjang menjadi segmen dengan durasi yang sesuai untuk pelatihan (biasanya 10-30 detik). Segmentasi meningkatkan efisiensi pelatihan dan mengurangi penggunaan memori.
4. **Silence Removal:** Penghapusan periode *silence* yang tidak informatif di awal dan akhir klip audio, sehingga model fokus pada bagian yang mengandung informasi linguistik.

5. **Quality Filtering:** Penyaringan klip dengan *Signal-to-Noise Ratio* (SNR) rendah atau distorsi signifikan untuk memastikan hanya data berkualitas tinggi yang digunakan dalam pelatihan.

2.2.6.2 Text Preprocessing

Transkrip teks juga memerlukan standardisasi:

1. **Normalisasi Huruf:** Konversi teks ke huruf kecil atau format konsisten lainnya untuk mengurangi variabilitas yang tidak relevan.
2. **Normalisasi Tanda Baca:** Standarisasi penggunaan tanda baca di seluruh dataset.
3. **Normalisasi Angka:** Konversi representasi numerik menjadi bentuk kata yang sesuai dengan cara angka diucapkan dalam bahasa target.
4. **Normalisasi Whitespace:** Pembersihan spasi berlebih dan karakter *whitespace* lainnya.

2.2.6.3 Data Augmentation

Data augmentation adalah teknik untuk meningkatkan keberagaman dan ukuran dataset pelatihan secara artifisial, yang sangat bermanfaat dalam skenario *low-resource*. Teknik augmentasi untuk ASR meliputi:

1. **SpecAugment** [21]: Augmentasi pada representasi spektrogram dengan menerapkan *masking* pada dimensi waktu dan frekuensi. Teknik ini memaksa model menjadi lebih *robust* terhadap variasi lokal dalam spektrum audio.
2. **Speed Perturbation** [22]: Variasi kecepatan audio dengan faktor tertentu (misalnya, 0.9, 1.0, 1.1) untuk mensimulasikan variasi kecepatan bicara alami.
3. **Noise Injection:** Penambahan *background noise* dengan SNR tertentu untuk meningkatkan *robustness* model terhadap kondisi audio yang beragam.

2.2.7 Regularisasi dan Optimisasi

Untuk mencegah *overfitting* pada dataset kecil dan memastikan konvergensi pelatihan yang stabil, berbagai teknik regularisasi dan optimisasi diterapkan.

2.2.7.1 Regularisasi

1. **Dropout:** Teknik regularisasi yang secara acak men-*drop* neuron selama pelatihan untuk mengurangi *co-adaptation* dan meningkatkan generalisasi.
2. **Label Smoothing:** Teknik yang menghaluskan distribusi target saat pelatihan untuk mencegah *overconfidence* pada prediksi dan meningkatkan kalibrasi model.
3. **Gradient Clipping:** Pembatasan norma gradien untuk mencegah *exploding gradients* selama *backpropagation*.
4. **Early Stopping:** Menghentikan pelatihan ketika performa pada *validation set* tidak lagi meningkat, mencegah *overfitting* pada *training set*.

2.2.7.2 Optimisasi

AdamW [23] adalah algoritma optimisasi yang digunakan secara luas untuk *fine-tuning* model *Transformer*. AdamW adalah varian dari Adam yang memisahkan *weight decay* dari optimisasi gradien, menghasilkan regularisasi yang lebih efektif. AdamW menggunakan *adaptive learning rates* per-parameter berdasarkan estimasi momen pertama dan kedua dari gradien, memungkinkan konvergensi cepat sambil mempertahankan stabilitas pelatihan.

2.2.8 K-Fold Cross-Validation

K-Fold Cross-Validation adalah teknik evaluasi model yang sangat penting untuk dataset berukuran kecil, di mana pembagian data tunggal (*fixed train-test split*) dapat menghasilkan estimasi performa yang bias atau tidak representatif [24]. Metode ini memaksimalkan penggunaan data yang tersedia dengan memastikan setiap sampel dalam dataset digunakan untuk *training* maupun *testing*.

2.2.8.1 Prinsip Dasar K-Fold Cross-Validation

Dalam *K-Fold Cross-Validation*, seluruh dataset dibagi menjadi K partisi (*folds*) dengan ukuran yang kurang lebih sama. Proses validasi dilakukan secara iteratif sebanyak K kali, di mana pada setiap iterasi:

1. Satu *fold* digunakan sebagai *test set*
2. $K - 1$ *folds* lainnya digabungkan sebagai *training set*
3. Model dilatih dari awal (*scratch*) pada *training set*
4. Model dievaluasi pada *test set* yang belum pernah dilihat
5. Metrik performa (misalnya, *accuracy*, WER, atau CER) dicatat

Setelah K iterasi, diperoleh K nilai metrik yang kemudian di-*aggregate* untuk menghasilkan estimasi performa final, biasanya dalam bentuk *mean $\pm$ standard deviation*.

2.2.8.2 Keuntungan untuk Dataset Kecil

Kohavi [24] menunjukkan bahwa *K-Fold Cross-Validation* memberikan estimasi performa yang lebih akurat dan *unbiased* dibandingkan *holdout validation* (pembagian tunggal), terutama untuk dataset kecil. Keuntungan utama meliputi:

- **Maximum Data Utilization:** Setiap sampel digunakan untuk *training* ($K - 1$ kali) dan *testing* (1 kali), memaksimalkan penggunaan data yang terbatas
- **Reduced Variance:** Dengan melakukan evaluasi pada K *folds* yang berbeda, variabilitas akibat pembagian data tertentu diminimalkan
- **Unbiased Performance Estimate:** Estimasi performa yang dihasilkan lebih mendekati performa sebenarnya pada data baru karena tidak bergantung pada satu pembagian data spesifik
- **Statistical Reliability:** Dengan K nilai metrik, dapat dihitung *confidence intervals* dan dilakukan *statistical significance testing* (misalnya, *paired t-test*) untuk membandingkan beberapa model

2.2.8.3 Pemilihan Nilai K

Nilai K yang umum digunakan adalah 5 atau 10. Pemilihan K melibatkan *trade-off*:

- **K kecil (e.g., 5):** Lebih cepat (fewer iterations), tetapi estimasi performa memiliki *variance* lebih tinggi
- **K besar (e.g., 10):** Estimasi lebih akurat dengan *variance* lebih rendah, tetapi memerlukan waktu komputasi lebih lama
- **Leave-One-Out CV ($K = N$):** Estimasi paling tidak bias, tetapi sangat mahal secara komputasi dan dapat memiliki *variance* tinggi

Untuk dataset *extremely low-resource* (misalnya, <200 sampel), nilai $K = 5$ sering menjadi pilihan yang seimbang antara akurasi estimasi dan biaya komputasi [24].

2.2.8.4 Stratified K-Fold

Untuk dataset dengan distribusi kelas atau karakteristik yang tidak seimbang, *Stratified K-Fold* memastikan bahwa setiap *fold* memiliki proporsi kelas yang sama dengan dataset lengkap. Pendekatan ini penting untuk menghindari bias evaluasi akibat distribusi data yang tidak representatif dalam beberapa *folds*.

2.2.8.5 Statistical Testing dengan Cross-Validation

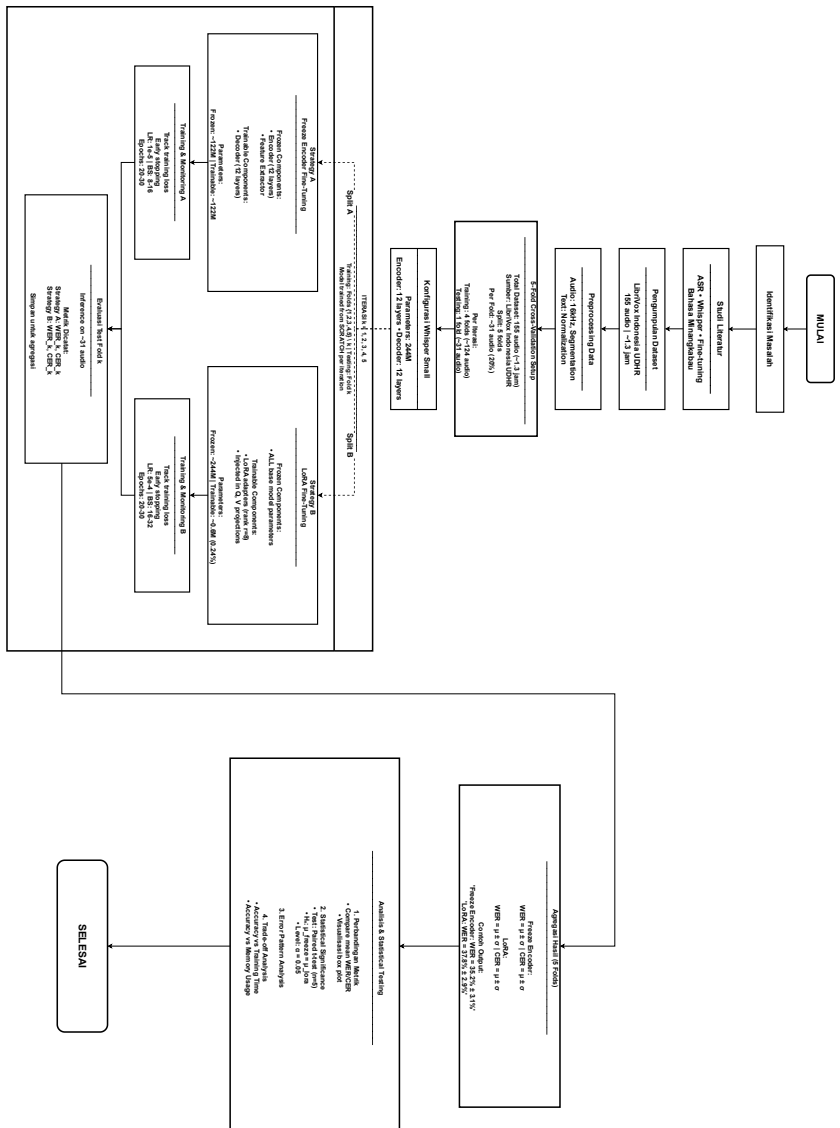
Salah satu keuntungan penting *K-Fold Cross-Validation* adalah kemampuan untuk melakukan *statistical significance testing* ketika membandingkan beberapa model. Dengan K *folds*, diperoleh K pasangan nilai metrik untuk setiap model yang dibandingkan. *Paired t-test* dapat diterapkan untuk menentukan apakah perbedaan performa antara dua model secara statistik signifikan ($p < 0.05$), memberikan kepercayaan yang lebih tinggi terhadap kesimpulan perbandingan model.

BAB III

METODE PENELITIAN

3.1 Alur Penelitian

Pada penelitian ini, alur dirancang untuk memastikan setiap tahapan pemrosesan dilakukan secara sistematis dan efisien. Alur penelitian ini mencerminkan langkah-langkah utama terkait bagaimana proses yang dilakukan dalam penelitian, dari awal sampai dengan akhir. Gambar dalam bentuk diagram alir (*flowchart*) seperti yang terlihat pada Gambar 3.1.



Gambar 3.1 Alur Penelitian Fine-Tuning Whisper untuk ASR Bahasa Minangkabau

3.2 Penjabaran Langkah Penelitian

Penelitian ini dilakukan secara sistematis melalui beberapa tahapan utama yang saling terkait. Setiap tahapan dirancang untuk memastikan model Whisper Small dapat diadaptasi secara optimal untuk mengenali bahasa Minangkabau. Berikut adalah penjabaran detail dari setiap langkah penelitian yang telah digambarkan pada flowchart di Subbab 3.1.

3.2.1 Identifikasi Masalah

Tahap pertama dalam alur penelitian ini adalah identifikasi masalah, yang dilakukan untuk memetakan konteks serta urgensi dari penelitian yang akan dilakukan. Proses ini diawali dengan analisis terhadap permasalahan di dunia nyata, yaitu rendahnya ketersediaan sistem *Automatic Speech Recognition* (ASR) untuk bahasa daerah Indonesia, khususnya bahasa Minangkabau. Analisis ini divalidasi dengan meninjau statistik dan studi terkait yang menunjukkan bahwa sebagian besar sistem ASR komersial yang tersedia saat ini dirancang untuk bahasa-bahasa dengan sumber daya tinggi seperti bahasa Inggris, Mandarin, dan bahasa Eropa lainnya, sementara bahasa daerah Nusantara masih sangat terbatas representasinya.

Setelah urgensi masalah teridentifikasi, proses dilanjutkan dengan menganalisis tantangan spesifik yang dihadapi dalam pengembangan ASR untuk bahasa *low-resource* seperti Minangkabau. Hasil analisis menemukan bahwa tantangan utama terletak pada minimnya ketersediaan dataset audio-transkrip berkualitas tinggi yang diperlukan untuk melatih model ASR. Kondisi ini berbeda dengan bahasa-bahasa besar yang memiliki ribuan jam data audio teranotasi. Identifikasi tantangan *low-resource* ini menegaskan kebutuhan akan pendekatan yang efisien, sehingga penelitian ini difokuskan pada pemanfaatan model *pre-trained* seperti Whisper yang telah dilatih pada skala besar dan dapat diadaptasi untuk bahasa target dengan data yang relatif terbatas.

Berdasarkan analisis tersebut, penelitian ini merumuskan *research gap*

yang jelas: belum ada penelitian yang secara komprehensif membandingkan strategi *fine-tuning* yang berbeda (*Freeze Encoder* vs LoRA) untuk adaptasi Whisper pada bahasa Minangkabau dalam skenario *extremely low-resource*. Identifikasi masalah ini mencakup beberapa aspek kunci berikut:

1. Analisis keterbatasan sistem ASR *existing* untuk bahasa daerah Indonesia, khususnya dari perspektif ketersediaan model dan dataset
2. Identifikasi tantangan bahasa *low-resource*: minimnya dataset audio-transkrip Minangkabau yang berkualitas dan terannotasi
3. Perumusan *research gap*: perlunya adaptasi model *pre-trained* (Whisper) untuk bahasa Minangkabau dengan pendekatan yang efisien secara komputasi
4. Penetapan objektif penelitian: membandingkan efektivitas strategi *fine-tuning* (*Freeze Encoder* vs LoRA) dalam hal akurasi dan efisiensi untuk dataset yang sangat terbatas
5. Penentuan metrik evaluasi utama: *Word Error Rate* (WER) dan *Character Error Rate* (CER) sebagai indikator kinerja model

3.2.2 Studi Literatur

Setelah masalah utama teridentifikasi, langkah selanjutnya dalam alur penelitian adalah melakukan studi literatur secara sistematis dan komprehensif. Proses ini bertujuan untuk memvalidasi *research gap* yang telah diidentifikasi pada tahap sebelumnya, mengumpulkan landasan teori yang relevan, serta memetakan metode *state-of-the-art* yang akan digunakan sebagai dasar perbandingan dalam penelitian ini. Studi literatur dilakukan dengan pendekatan yang terstruktur untuk memastikan bahwa semua aspek penting dari penelitian tercakup dengan baik.

Studi literatur difokuskan pada empat area utama yang saling berkaitan. Pertama, mendalami arsitektur dan kemampuan model Whisper dari OpenAI, termasuk mekanisme *encoder-decoder Transformer*, proses *pre-training* pada

680,000 jam data multibahasa, dan karakteristik yang membuat Whisper unggul dalam skenario *zero-shot* maupun *few-shot*. Kedua, mengidentifikasi dan menganalisis strategi *fine-tuning* yang telah terbukti efektif untuk skenario *extremely low-resource*, dengan fokus khusus pada perbandingan antara *Freeze Encoder* yang hanya melatih *decoder* versus teknik *Parameter-Efficient Fine-Tuning* (PEFT) seperti LoRA yang hanya melatih sebagian kecil parameter. Ketiga, mempelajari penelitian-penelitian terdahulu yang telah menerapkan Whisper atau model ASR lainnya pada bahasa Indonesia dan bahasa daerah Nusantara, untuk memahami tantangan spesifik dan solusi yang telah dicoba. Keempat, memahami karakteristik linguistik bahasa Minangkabau, termasuk fonologi, morfologi, dan pola ucapan yang dapat mempengaruhi performa model ASR.

Proses studi literatur dilaksanakan melalui tahapan-tahapan berikut:

1. *Review* paper ilmiah terkait Whisper dan strategi *fine-tuning* dari jurnal internasional bereputasi (e.g., ICASSP, Interspeech, ACL) dan *conference proceedings* di bidang *speech processing*
2. Analisis mendalam terhadap penelitian terdahulu tentang ASR untuk bahasa Indonesia dan bahasa daerah Nusantara, dengan fokus pada metodologi, dataset yang digunakan, dan hasil yang dicapai
3. Studi dokumentasi teknis OpenAI Whisper dan ekosistem Hugging Face Transformers *library*, termasuk *API reference*, *model cards*, dan *best practices* untuk *fine-tuning*
4. Eksplorasi metode *fine-tuning* yang tersedia, membandingkan karakteristik *Freeze Encoder* dengan pendekatan PEFT seperti LoRA, Adapter, dan Prefix-Tuning
5. Pemahaman mendalam tentang metrik evaluasi ASR, khususnya *Word Error Rate* (WER) dan *Character Error Rate* (CER), termasuk interpretasi dan keterbatasannya

Hasil dari studi literatur ini menjadi landasan teoritis yang kokoh dalam

memilih arsitektur model, menentukan strategi *fine-tuning* yang optimal, dan merancang metodologi evaluasi yang tepat untuk penelitian ini. Selain itu, studi literatur juga mengonfirmasi validitas *research gap* yang telah diidentifikasi dan memberikan justifikasi ilmiah untuk pendekatan yang diusulkan.

3.2.3 Pengumpulan Dataset

Penelitian ini menggunakan dataset publik LibriVox Indonesia versi 1.0, yang merupakan koleksi rekaman audio dari buku-buku domain publik dalam berbagai bahasa daerah Indonesia. Secara spesifik, dataset bahasa Minangkabau yang digunakan berisi rekaman pembacaan *Universal Declaration of Human Rights* (Pernyataan Umum tentang Hak-Hak Asasi Manusia) dalam bahasa Minangkabau. Dataset ini termasuk kategori *extremely low-resource* dengan total durasi sekitar **1.3 jam audio**, terdiri dari **155 segmen audio** dengan durasi rata-rata 30 detik per segmen.

Pemilihan dataset LibriVox Indonesia sebagai sumber data didasarkan pada beberapa pertimbangan penting. Pertama, LibriVox menyediakan audio dengan kualitas rekaman yang relatif baik karena mengikuti standar produksi *audiobook* dengan *sampling rate* 44.1 kHz. Kedua, setiap audio dilengkapi dengan transkrip teks yang akurat karena dibacakan dari naskah tertulis (teks UDHR), sehingga mengurangi beban anotasi manual dan menjamin akurasi transkrip. Ketiga, dataset ini tersedia secara publik dengan lisensi *Public Domain* (CC-0), memungkinkan penggunaan bebas untuk penelitian akademis. Namun, penting untuk dicatat bahwa dataset terbatas pada domain **pembacaan formal** (*formal reading*) dari teks legal/deklarasi, sehingga performa model pada percakapan spontan atau konteks informal mungkin berbeda.

Proses pengumpulan dataset dirancang secara sistematis dengan menggabungkan seluruh data yang tersedia dari folder *train* dan *test* yang disediakan oleh *maintainer* Hugging Face. Tahap awal dimulai dengan mengunduh dataset bahasa Minangkabau dari repositori LibriVox Indonesia

melalui platform Hugging Face Datasets. Setelah dataset diunduh, dilakukan proses verifikasi kualitas untuk memastikan bahwa setiap segmen audio memiliki kejernihan ucapan yang baik (*clear speech*), minim *background noise*, dan tidak mengandung distorsi audio yang signifikan. Dataset LibriVox Indonesia telah melalui proses segmentasi otomatis yang membagi rekaman panjang menjadi klip-klip pendek dengan durasi beberapa detik hingga maksimal 30 detik, yang ideal untuk pelatihan model ASR.

Proses pengumpulan dan persiapan dataset mencakup langkah-langkah berikut:

1. Mengunduh dataset LibriVox Indonesia 1.0 dari Hugging Face Datasets, dengan fokus pada subset bahasa Minangkabau yang berisi pembacaan UDHR
2. Menggabungkan seluruh audio dari folder *train* dan *test* yang disediakan *maintainer* untuk memaksimalkan penggunaan data dalam skema *cross-validation*
3. Verifikasi kualitas audio secara teknis untuk setiap segmen: *clear speech*, minimal *background noise*, tidak ada distorsi atau *clipping*
4. Validasi transkrip teks yang sesuai dengan setiap segmen audio, memastikan akurasi dan kelengkapan transkrip berdasarkan teks UDHR dalam bahasa Minangkabau
5. Ekstraksi metadata *speaker (reader ID)* untuk dokumentasi keberagaman pembicara dalam dataset
6. Inventarisasi total durasi audio yang tersedia (155 segmen @ 30s = 1.3 jam total)

Dataset yang berhasil dikumpulkan mencakup rekaman pembacaan UDHR dalam bahasa Minangkabau dengan total 1.3 jam audio (155 segmen). Meskipun terbatas pada satu jenis teks (dokumen deklarasi HAM) dan durasi yang sangat kecil, dataset ini tetap merepresentasikan bahasa Minangkabau formal dan standar. Karakteristik *extremely low-resource* ini membuat penelitian

ini mengadopsi pendekatan **5-Fold Cross-Validation** untuk memaksimalkan penggunaan data dan memperoleh evaluasi yang lebih robust, dengan strategi *transfer learning* yang sangat efisien dari model *pre-trained* Whisper.

3.2.4 Preprocessing Dataset

Dataset mentah yang telah dikumpulkan akan melalui tahapan *preprocessing* yang sistematis dan komprehensif untuk memastikan kompatibilitas dengan persyaratan teknis model Whisper serta mengoptimalkan kualitas data yang akan digunakan dalam proses *fine-tuning*. *Preprocessing* merupakan tahap kritis dalam penelitian ini karena kualitas dan konsistensi data input secara langsung mempengaruhi kinerja model ASR yang dilatih. Model Whisper memiliki spesifikasi teknis tertentu, terutama terkait format audio input, yang harus dipenuhi agar model dapat berfungsi secara optimal. Selain itu, *preprocessing* juga bertujuan untuk standardisasi data sehingga mengurangi variabilitas yang tidak relevan dan memungkinkan model fokus pada pembelajaran pola linguistik yang penting.

Proses *preprocessing* dirancang dalam tiga tahap utama yang saling melengkapi: *audio preprocessing*, *text preprocessing*, dan *dataset splitting*. Setiap tahap memiliki tujuan spesifik dan menggunakan teknik-teknik yang telah terbukti efektif dalam penelitian ASR sebelumnya. Pendekatan sistematis ini memastikan bahwa dataset yang dihasilkan memiliki kualitas yang konsisten, kompatibel dengan arsitektur Whisper, dan siap digunakan untuk proses *training*, *validation*, dan *testing*.

3.2.4.1 Audio Preprocessing

Tahap *audio preprocessing* berfokus pada transformasi sinyal audio mentah menjadi format yang sesuai dengan spesifikasi teknis Whisper dan mengoptimalkan kualitas sinyal untuk pembelajaran model. Proses ini mencakup beberapa operasi penting:

1. **Format Conversion:** Konversi format audio dari MP3 (format asli LibriVox Indonesia) ke WAV (*Waveform Audio File Format*). Konversi ini diperlukan karena format MP3 yang terkompresi dengan *lossy compression* memerlukan proses *decoding* tambahan yang dapat membebani komputasi Whisper. Format WAV yang *uncompressed* memungkinkan akses langsung ke data audio mentah (*raw audio samples*), sehingga mengurangi beban komputasi dan mempercepat proses *preprocessing* serta *training*
2. **Resampling:** Konversi *sampling rate* audio dari 44.1 kHz (format asli LibriVox Indonesia) ke 16 kHz, yang merupakan spesifikasi standar yang digunakan oleh model Whisper. Konversi ini penting karena model Whisper dilatih dengan data audio pada frekuensi 16 kHz dan akan menghasilkan performa optimal pada *sampling rate* yang sama
3. **Segmentation:** Pemotongan audio panjang menjadi *clips* dengan durasi 10-30 detik untuk meningkatkan efisiensi *training* dan mengurangi penggunaan memori GPU. Segmentasi juga membantu model mempelajari pola dalam konteks yang lebih terfokus
4. **Normalization:** Normalisasi *amplitude* audio untuk memastikan konsistensi volume di seluruh dataset, menghindari bias model terhadap audio dengan volume tertentu
5. **Silence Removal:** Penghapusan periode *silence* yang tidak informatif di awal dan akhir setiap *clip*, sehingga model tidak perlu memproses bagian audio yang tidak mengandung informasi linguistik
6. **Quality Check:** Penyaringan *clips* dengan *Signal-to-Noise Ratio* (SNR) rendah atau mengandung distorsi signifikan, untuk memastikan hanya data berkualitas tinggi yang digunakan dalam *training*
7. **SpecAugment:** Teknik *augmentation* pada representasi spektrogram dengan menerapkan *time masking* (T=70) dan *frequency masking* (F=27) untuk meningkatkan invariansi temporal dan frekuensi, diterapkan selama *training* untuk meningkatkan *robustness* model

8. **Speed Perturbation:** Variasi kecepatan audio dengan faktor $\{0.9, 1.0, 1.1\}$ untuk mensimulasikan variasi kecepatan bicara alami dan mencegah *overfitting* pada pola temporal yang spesifik
9. **Noise Injection:** Penambahan *environmental noise* dengan SNR 15-25 dB untuk meningkatkan *robustness* model terhadap kondisi audio yang beragam dan meningkatkan generalisasi pada skenario *low-resource*

3.2.4.2 Text Preprocessing

Tahap *text preprocessing* bertujuan untuk menstandarkan transkrip teks dan menghilangkan variasi yang tidak relevan untuk tugas ASR. Standarisasi teks penting untuk memastikan konsistensi dalam pelatihan model dan evaluasi hasil. Proses ini mencakup:

1. **Lowercase Conversion:** Konversi semua teks ke huruf kecil untuk mengurangi ukuran *vocabulary* dan menghilangkan perbedaan yang tidak relevan antara kapitalisasi
2. **Punctuation Normalization:** Standarisasi tanda baca untuk konsistensi format, termasuk normalisasi tanda kutip, tanda hubung, dan simbol-simbol khusus
3. **Number Normalization:** Konversi representasi numerik menjadi bentuk kata (*e.g.*, "10" → "sepuluh", "2025" → "dua ribu dua puluh lima") untuk konsistensi dengan bagaimana angka diucapkan dalam bahasa Minangkabau
4. **Whitespace Normalization:** Pembersihan spasi berlebih, tab, dan karakter *whitespace* lainnya untuk memastikan format teks yang konsisten
5. **Special Character Handling:** Penanganan khusus untuk karakter-karakter yang spesifik dalam bahasa Minangkabau, termasuk diakritik atau simbol yang mungkin muncul dalam transkrip

3.2.4.3 5-Fold Cross-Validation

Untuk memaksimalkan penggunaan data yang sangat terbatas dan memperoleh evaluasi yang lebih robust, penelitian ini menggunakan pendekatan **5-Fold Cross-Validation**. Seluruh 155 segmen audio yang tersedia (dari gabungan folder *train* dan *test* yang disediakan *maintainer* Hugging Face) dibagi menjadi 5 *folds* dengan distribusi yang seimbang. Proses *cross-validation* dilakukan sebagai berikut:

1. **Fold Splitting:** Dataset dibagi menjadi 5 *folds*, masing-masing berisi sekitar 31 audio (~20% dari total)
2. **Iterative Training:** Pada setiap iterasi (total 5 iterasi):
 - Satu *fold* digunakan sebagai *test set* (~31 audio, ~0.26 jam)
 - Empat *folds* lainnya digabungkan sebagai *training set* (~124 audio, ~1.03 jam)
 - Model dilatih dari *scratch* (dari *pre-trained* Whisper Small) pada *training set*
 - Model dievaluasi pada *test set* yang belum pernah dilihat
3. **Aggregation:** Hasil WER dan CER dari 5 iterasi di-*aggregate* untuk menghasilkan metrik final berupa *mean $\pm$ standard deviation*

Pendekatan *5-fold cross-validation* memberikan beberapa keuntungan penting untuk dataset *extremely low-resource*:

- **Evaluasi lebih robust:** Setiap audio memiliki kesempatan untuk menjadi bagian dari *test set*, mengurangi bias akibat pembagian data yang tidak representative
- **Maximum data utilization:** Seluruh 155 audio digunakan untuk *training* DAN *testing*, tidak ada data yang "terbuang"
- **Statistical reliability:** Dengan 5 *folds*, diperoleh 5 nilai WER/CER yang dapat digunakan untuk menghitung *mean* dan *standard deviation*, memberikan gambaran variabilitas performa model

- **Fair comparison:** Kedua strategi (*Freeze Encoder* dan LoRA) dievaluasi pada *folds* yang identik, memastikan perbandingan yang adil

Dengan pendekatan ini, penelitian tidak menggunakan *validation set* terpisah. Monitoring selama *training* dilakukan dengan memantau *training loss* dan menerapkan *early stopping* berbasis penurunan *loss* untuk mencegah *overfitting*.

3.2.5 Konfigurasi Model Whisper Small

Model Whisper Small dipilih sebagai arsitektur *base model* dalam penelitian ini berdasarkan pertimbangan yang cermat terhadap *trade-off* antara performa, efisiensi komputasi, dan kelayakan untuk skenario *low-resource*. Whisper Small, dengan 244 juta parameter, merupakan varian menengah dari keluarga model Whisper yang menawarkan keseimbangan optimal: cukup besar untuk menangkap kompleksitas bahasa tetapi tidak terlalu besar sehingga memerlukan sumber daya komputasi yang tidak realistis untuk *fine-tuning* pada infrastruktur *cloud* yang tersedia seperti RunPod. Pemilihan ini juga didukung oleh studi-studi sebelumnya yang menunjukkan bahwa Whisper Small dapat mencapai performa yang kompetitif pada berbagai tugas ASR, terutama setelah di-*fine-tune* pada bahasa target.

Arsitektur Whisper Small menggunakan *encoder-decoder Transformer* dengan konfigurasi yang telah dioptimalkan untuk tugas *speech recognition*. Model ini telah di-*pre-train* pada dataset multibahasa yang sangat besar (680,000 jam audio) sehingga memiliki representasi yang kuat tentang pola akustik dan linguistik lintas bahasa. Karakteristik *pre-trained* ini memungkinkan model untuk di-*transfer* dengan efisien ke bahasa Minangkabau meskipun dengan data *fine-tuning* yang relatif terbatas (10-20 jam). Proses konfigurasi model dilakukan secara sistematis untuk memastikan semua komponen berfungsi dengan benar dan siap untuk tahap *fine-tuning*.

Tahap konfigurasi model Whisper Small mencakup beberapa langkah

teknis penting:

1. *Loading pre-trained* Whisper Small (244M *parameters*) dari Hugging Face Model Hub menggunakan *library* Transformers, memastikan model dalam keadaan *pre-trained* yang telah terbukti efektif
2. Verifikasi arsitektur model: 12 *encoder layers* dan 12 *decoder layers*, dengan *hidden dimension (width)* 768 dan 12 *attention heads* per *layer*, sesuai dengan spesifikasi resmi Whisper Small
3. Inisialisasi *tokenizer* dengan *vocabulary* sebesar 51,864 *tokens*, yang mencakup representasi karakter dan subword untuk berbagai bahasa termasuk kemampuan untuk mengakomodasi bahasa baru
4. Konfigurasi *feature extractor* untuk menghasilkan representasi *log-Mel spectrogram* dengan 80 dimensi, menggunakan *window size* 25ms dan *hop length* 10ms, yang merupakan parameter standar untuk ekstraksi fitur audio dalam Whisper
5. *Setup device* (GPU/CPU) untuk *training*, dengan preferensi GPU (NVIDIA GeForce RTX 4090 dengan 24 GB VRAM) untuk mempercepat komputasi matriks dalam proses *fine-tuning*

3.2.6 Implementasi Fine-Tuning

Penelitian ini menerapkan pendekatan komparatif dengan membandingkan dua strategi *fine-tuning* yang berbeda: *Freeze Encoder Fine-Tuning* dan *Low-Rank Adaptation* (LoRA). Pemilihan kedua strategi ini didasarkan pada karakteristik yang saling melengkapi dalam konteks *extremely low-resource*. *Freeze Encoder* merupakan pendekatan yang membekukan seluruh *encoder* dan hanya melatih *decoder*, mempertahankan representasi akustik universal yang telah dipelajari selama *pre-training* sambil mengadaptasi komponen linguistik untuk bahasa Minangkabau. Sebaliknya, LoRA merupakan teknik *Parameter-Efficient Fine-Tuning* (PEFT) yang hanya melatih sebagian kecil parameter tambahan ($\sim 0.24\%$ dari total), menawarkan efisiensi komputasi yang maksimal.

Implementasi kedua metode dilakukan secara terpisah dan sistematis untuk memastikan perbandingan yang adil dan valid.

Pendekatan komparatif ini memungkinkan analisis mendalam terhadap *trade-off* antara akurasi model dan efisiensi sumber daya. Dalam konteks bahasa Minangkabau dengan dataset *extremely low-resource* (1.3 jam), pertanyaan penting yang perlu dijawab adalah: apakah efisiensi LoRA (dengan hanya melatih $\sim 0.24\%$ parameter) memberikan hasil yang sebanding dengan *Freeze Encoder* (yang melatih 50% parameter, yaitu seluruh *decoder*)? Jika LoRA dapat mencapai performa yang mendekati *Freeze Encoder* dengan biaya komputasi yang jauh lebih rendah, maka LoRA menjadi solusi yang lebih praktis dan dapat diskalakan untuk bahasa-bahasa *extremely low-resource* lainnya. Untuk menjawab pertanyaan ini, implementasi kedua metode dirancang dengan cermat dengan pengaturan *hyperparameter* yang dioptimalkan untuk masing-masing pendekatan.

3.2.6.1 Freeze Encoder Fine-Tuning

Pada metode *Freeze Encoder*, seluruh parameter *encoder* Whisper Small (yang berfungsi memproses input audio) dibekukan (*frozen*), dan hanya parameter *decoder* (yang menghasilkan teks transkrip) yang di-*update* selama proses *training*. Pendekatan ini sangat sesuai untuk skenario *extremely low-resource* karena mempertahankan representasi akustik universal yang telah dipelajari *encoder* dari 680,000 jam data *pre-training*, sementara hanya mengadaptasi *decoder* untuk pola linguistik bahasa Minangkabau. Dengan hanya melatih *decoder* (122 juta dari 244 juta parameter total), pendekatan ini mengurangi risiko *overfitting* yang sangat tinggi pada dataset 1.3 jam.

Konfigurasi *training* untuk *Freeze Encoder* adalah sebagai berikut:

1. **Frozen Components:** Seluruh 12 lapisan *encoder* Transformer dan *feature extractor* (konvolusi) dibekukan
2. **Trainable Components:** Seluruh 12 lapisan *decoder* Transformer

(~122M parameter)

3. **Optimizer:** AdamW (*Adam with Weight Decay*) dengan *weight decay* 0.01 untuk regularisasi
4. **Learning Rate:** 1×10^{-5} (disesuaikan untuk melatih *decoder* dengan stabil)
5. **Batch Size:** 8-16 sampel per *batch*, disesuaikan dengan memori GPU
6. **Epochs:** 20-30 *epochs* dengan mekanisme *early stopping* untuk mencegah *overfitting*
7. **Loss Function:** *Cross-entropy loss* standar untuk tugas klasifikasi sekuens

3.2.6.2 LoRA Fine-Tuning

Metode LoRA (*Low-Rank Adaptation*) bekerja dengan menambahkan matriks *low-rank* sebagai parameter yang dapat dilatih pada *attention layers*, sementara seluruh parameter *pre-trained* dari Whisper Small tetap *frozen*. Pendekatan ini sangat efisien karena hanya melatih ~2% dari total parameter (sekitar 5 juta parameter dari 244 juta), namun tetap mampu mencapai performa yang kompetitif. Konfigurasi LoRA *fine-tuning* adalah sebagai berikut:

1. **LoRA Configuration:**
 - *Rank (r)*: 8 (dimensi matriks *low-rank* yang ditambahkan)
 - *Alpha*: 16 (*scaling factor* untuk LoRA updates)
 - *Dropout*: 0.1 (regularisasi pada LoRA layers)
 - *Target modules*: *Query* dan *Value projections* dalam *attention layers*, yang merupakan komponen paling kritis untuk adaptasi
2. **Trainable Parameters:** ~2% dari total parameter (~5 juta parameter), drastis lebih sedikit dibanding *Freeze Encoder*
3. **Optimizer:** AdamW dengan *weight decay* 0.01
4. **Learning Rate:** 5×10^{-4} (lebih tinggi dibanding *Freeze Encoder* karena hanya melatih sebagian kecil parameter pada *frozen base model*)
5. **Batch Size:** 16 sampel per *batch*

6. **Epochs:** 10-15 *epochs* dengan *early stopping* (*patience*=5)

3.2.6.3 Regularization

Selain *data augmentation*, beberapa teknik *regularization* juga diterapkan untuk mencegah *overfitting* dan meningkatkan generalisasi model:

1. *Dropout* pada *attention layers* ($p=0.1$) dan *feed-forward network* (FFN) *layers* ($p=0.3$) untuk mengurangi *co-adaptation* antar *neurons*
2. *Label smoothing* ($\epsilon = 0.1$) untuk mencegah overconfidence pada prediksi dan meningkatkan kalibrasi model
3. *Gradient clipping* ($\text{max norm} = 1.0$) untuk mencegah *exploding gradients* selama *training*
4. *Early stopping* berdasarkan *validation WER*, menghentikan *training* jika tidak ada perbaikan selama 5 *epochs* berturut-turut

3.2.7 Training dan Monitoring

Proses *training* dilakukan dengan protokol *monitoring* yang ketat dan sistematis untuk memastikan konvergensi yang optimal serta mendeteksi potensi masalah sejak dini. *Monitoring* yang komprehensif sangat penting dalam penelitian ini mengingat kompleksitas model Whisper dan sifat dataset yang *low-resource*. Tanpa *monitoring* yang tepat, berbagai masalah seperti *overfitting*, konvergensi yang lambat, atau konfigurasi *hyperparameter* yang tidak optimal dapat tidak terdeteksi dan menghasilkan model dengan performa suboptimal. Oleh karena itu, strategi *monitoring* dirancang untuk memberikan visibilitas penuh terhadap seluruh aspek proses *training*, dari metrik performa hingga penggunaan sumber daya komputasi.

Infrastruktur *training* menggunakan RunPod *Cloud GPU Service* dengan NVIDIA GeForce RTX 4090 (24 GB VRAM), yang menyediakan performa komputasi tinggi untuk *fine-tuning* model berukuran menengah seperti Whisper Small. Seluruh proses *training* untuk kedua strategi (*Freeze Encoder* dan

LoRA) dilakukan pada infrastruktur yang sama untuk memastikan perbandingan yang adil. Metrik-metrik kunci dicatat secara otomatis menggunakan *logging framework* yang terintegrasi, dan *checkpointing* dilakukan secara berkala untuk menyimpan *state* model terbaik.

Protokol *training* dan *monitoring* yang diterapkan mencakup aspek-aspek berikut:

1. *Training* dilakukan pada RunPod dengan akselerasi GPU NVIDIA GeForce RTX 4090 (24 GB VRAM) untuk mempercepat komputasi matriks dalam *backpropagation*
2. *Logging* metrik setiap 100 *steps*: *training loss*, *learning rate* saat ini, dan waktu komputasi per *batch*, untuk memantau progres *training* secara real-time
3. Evaluasi pada *validation set* setiap *epoch*: perhitungan WER, CER, dan *validation loss* untuk memantau kemampuan generalisasi model
4. *Checkpoint* model terbaik berdasarkan *lowest validation WER*, memastikan model yang disimpan adalah yang paling optimal untuk tugas ASR bahasa Minangkabau
5. *Tracking training time* dan penggunaan memori GPU untuk analisis efisiensi komputasi, terutama penting untuk membandingkan *Freeze Encoder* dengan LoRA
6. Visualisasi *learning curves* (*loss*, WER, CER vs *epochs*) menggunakan *TensorBoard* atau *Weights & Biases* untuk analisis visual terhadap konvergensi model

3.2.8 Evaluasi Model

Model yang telah di-*fine-tune* dievaluasi secara komprehensif menggunakan *test set* yang tidak pernah dilihat oleh model selama proses *training*. Prinsip isolasi *test set* ini sangat penting untuk memastikan bahwa metrik evaluasi yang diperoleh merepresentasikan kemampuan generalisasi

model yang sebenarnya pada data baru, bukan hanya kemampuan menghafal pola dalam data *training*. Evaluasi dilakukan dengan protokol yang konsisten untuk kedua strategi *fine-tuning* (*Freeze Encoder* dan LoRA) untuk memungkinkan perbandingan yang adil dan valid melalui skema *5-fold cross-validation*.

Proses evaluasi dirancang untuk mengukur berbagai aspek performa model, tidak hanya akurasi transkrip tetapi juga *error patterns* dan karakteristik kesalahan yang terjadi. Analisis *error patterns* memberikan *insights* penting tentang jenis kesalahan yang paling sering dibuat model (substitusi, penghapusan, atau penyisipan kata), yang dapat menjadi dasar untuk perbaikan model di masa depan.

Protokol evaluasi yang diterapkan mencakup tahapan-tahapan berikut:

1. *Inference* pada *test set* menggunakan algoritma *beam search* dengan *beam size* = 5, yang memberikan keseimbangan antara kualitas transkrip dan kecepatan inferensi
2. Perhitungan *Word Error Rate* (WER) sebagai metrik utama untuk mengukur akurasi transkrip pada level kata
3. Perhitungan *Character Error Rate* (CER) sebagai metrik tambahan yang lebih sensitif terhadap kesalahan fonetik dan memberikan perspektif komplementer terhadap WER
4. Normalisasi teks sebelum evaluasi (konversi ke *lowercase*, penghapusan tanda baca, normalisasi *whitespace*) untuk memastikan konsistensi dalam perhitungan metrik
5. Perhitungan metrik untuk setiap *fold*: WER dan CER pada *test set* dari masing-masing *5 folds*
6. Agregasi hasil: Menghitung *mean ± standard deviation* dari WER dan CER across *5 folds* untuk kedua strategi (*Freeze Encoder* dan LoRA)
7. Analisis *error patterns* detail dengan mengkategorikan kesalahan menjadi: substitusi (kata yang salah diganti), penghapusan (kata yang hilang), dan penyisipan (kata tambahan yang tidak seharusnya ada)

8. Perbandingan performa antara model *Freeze Encoder* dan LoRA menggunakan metrik agregat (mean WER/CER), serta analisis *trade-off* dengan waktu *training* dan penggunaan memori

3.2.9 Analisis Hasil dan Perbandingan

Tahap akhir dari alur penelitian adalah melakukan analisis mendalam terhadap hasil evaluasi dan membandingkan performa antara dua strategi *fine-tuning* yang telah diimplementasikan. Analisis ini tidak hanya berfokus pada perbandingan metrik akurasi (WER dan CER), tetapi juga mempertimbangkan aspek-aspek praktis lainnya seperti efisiensi komputasi, waktu *training*, penggunaan memori, dan kelayakan untuk deployment. Pendekatan analisis komparatif ini bertujuan untuk memberikan rekomendasi yang komprehensif dan berbasis bukti mengenai strategi *fine-tuning* yang paling sesuai untuk adaptasi Whisper pada bahasa Minangkabau dan bahasa *low-resource* lainnya.

Analisis dilakukan dengan mengintegrasikan evaluasi kuantitatif (metrik numerik) dan kualitatif (analisis transkrip), memberikan pemahaman yang holistik tentang kekuatan dan kelemahan masing-masing strategi. Evaluasi kuantitatif mencakup perbandingan statistik WER dan CER, sementara evaluasi kualitatif melibatkan inspeksi manual terhadap sampel transkrip untuk mengidentifikasi pola kesalahan yang mungkin tidak tertangkap oleh metrik numerik. Pendekatan ganda ini memastikan bahwa kesimpulan yang ditarik tidak hanya berdasarkan angka-angka tetapi juga didukung oleh pemahaman mendalam tentang perilaku model.

Proses analisis hasil dan perbandingan mencakup aspek-aspek berikut:

1. Perbandingan metrik WER dan CER antara *Freeze Encoder* dan LoRA berdasarkan $\text{mean} \pm \text{standard deviation}$ dari 5 folds
2. **Statistical significance testing:** Menerapkan *paired t-test* pada hasil WER/CER dari 5 folds untuk menentukan apakah perbedaan antara *Freeze Encoder* dan LoRA secara statistik signifikan ($p\text{-value} < 0.05$)

3. Analisis *trade-off* multidimensional: akurasi model vs efisiensi *training* (waktu pelatihan total untuk 5 *folds*, penggunaan memori GPU, jumlah parameter yang dilatih), untuk mengidentifikasi strategi yang menawarkan keseimbangan terbaik untuk skenario *extremely low-resource*
4. Evaluasi kualitas transkrip secara kualitatif melalui inspeksi manual terhadap sampel transkrip dari berbagai *folds*, mengidentifikasi jenis kesalahan yang paling umum dan konteks di mana setiap metode unggul atau lemah
5. Identifikasi *patterns of errors* yang spesifik (*common misrecognitions*), seperti fonem atau kata Minangkabau yang sering salah dikenali, untuk memberikan *insights* bagi penelitian lanjutan
6. Diskusi tentang *limitations* penelitian (*e.g.*, ukuran dataset yang sangat terbatas, domain terbatas pada pembacaan formal UDHR, computational cost dari 5-fold training) dan *potential improvements* untuk penelitian masa depan
7. Perumusan rekomendasi strategi *fine-tuning* terbaik untuk bahasa Minangkabau dan bahasa *extremely low-resource* lainnya berdasarkan hasil analisis komprehensif dari *cross-validation*

3.3 Alat dan Bahan Tugas Akhir

Penelitian ini memerlukan berbagai alat dan bahan untuk mendukung implementasi dan evaluasi model ASR bahasa Minangkabau.

3.3.1 Alat

Alat-alat yang digunakan dalam penelitian ini meliputi perangkat keras dan perangkat lunak:

3.3.1.1 Perangkat Keras

1. **Laptop:** ASUS VivoBook 14 K413 (11th Gen Intel), digunakan untuk *development*, analisis data, dan penulisan kode

- Processor: Intel Core i3 (11th Generation)
 - RAM: 8 GB DDR4
 - Storage: 512 GB SSD
 - Sistem operasi: Linux/Windows
 - Fungsi: *Development environment*, eksplorasi data, dan analisis hasil
2. **RunPod Cloud GPU Service:** Platform *cloud computing* untuk *training* model dengan GPU
- GPU: NVIDIA GeForce RTX 4090 (24 GB VRAM)
 - RAM: 36 GB
 - vCPU: 8 *virtual cores*
 - Storage: 80 GB *disk space*
 - Container: `runpod/pytorch:1.0.2-cu1281-torch280-ubuntu2404`
 - Fungsi: *Training* model *Freeze Encoder* dan LoRA dengan akselerasi GPU

3.3.1.2 Perangkat Lunak

1. **Python 3.10+:** Bahasa pemrograman utama untuk implementasi model dan *data processing*
2. **PyTorch 2.8.0:** *Deep learning framework* untuk *training* dan *inference* model Whisper
3. **CUDA 12.8.1:** *NVIDIA CUDA Toolkit* untuk akselerasi GPU pada RunPod
4. **Ubuntu 24.04 LTS:** Sistem operasi *Linux* pada *container* RunPod (`runpod/pytorch:1.0.2-cu1281-torch280-ubuntu2404`)
5. **Hugging Face Transformers:** *Library* untuk memuat dan *fine-tune* model Whisper *pre-trained*
6. **Hugging Face Datasets:** *Library* untuk manajemen dataset, *data loading*, dan *streaming* dari Hugging Face Hub
7. **Hugging Face Accelerate:** *Library* untuk *distributed training*, optimisasi

memori, dan akselerasi proses *training* pada GPU

8. **Hugging Face Evaluate:** *Library* untuk evaluasi metrik model *machine learning* secara standar dan konsisten
9. **PEFT (Parameter-Efficient Fine-Tuning):** *Library* dari Hugging Face untuk implementasi LoRA dan metode *fine-tuning* yang efisien parameter
10. **torchaudio:** *Library* PyTorch untuk *audio processing*, transformasi, dan augmentasi data audio
11. **torchcodec:** *Library* untuk *encoding* dan *decoding* berbagai format audio dengan *backend* PyTorch
12. **audiomentations:** *Library* untuk *data augmentation* audio (*noise injection, pitch shifting, time stretching*)
13. **jiwer:** *Library* untuk perhitungan metrik evaluasi ASR (WER dan CER)
14. **scikit-learn:** *Library* untuk utilitas *machine learning* (*data splitting, metrik evaluasi, preprocessing*)
15. **TensorBoard:** Platform visualisasi untuk *monitoring* metrik *training*, *loss curves*, dan parameter model secara *real-time*
16. **Weights & Biases (wandb):** Platform untuk *experiment tracking*, visualisasi *learning curves*, perbandingan eksperimen, dan *logging* metrik
17. **Jupyter Notebook:** *Interactive development environment* untuk eksplorasi data, eksperimen model, dan dokumentasi proses penelitian
18. **Git/GitHub:** *Version control system* dan repositori kode untuk manajemen proyek dan kolaborasi
19. **Visual Studio Code:** *Code editor* untuk *local development* pada ASUS VivoBook 14 K413

3.3.2 Bahan

Bahan-bahan yang digunakan dalam penelitian ini meliputi dataset dan dokumen referensi:

1. **Dataset Audio Bahasa Minangkabau:**

- Sumber: LibriVox Indonesia 1.0 via Hugging Face Datasets (<https://huggingface.co/datasets/indonesian-nlp/librivox-indonesia>)
- Konten: Pembacaan *Universal Declaration of Human Rights* dalam bahasa Minangkabau
- Format: Audio files MP3 dengan *sampling rate* 44.1 kHz
- Durasi: Total 1.3 jam (155 audio @ 30 detik per audio)
- Evaluasi: 5-Fold Cross-Validation (setiap fold: 124 train, 31 test)
- Karakteristik: *Extremely low-resource*, domain pembacaan formal (UDHR)
- Lisensi: *Public Domain* (CC-0)

2. Model Pre-trained:

- Whisper Small dari OpenAI (via Hugging Face Model Hub)
- Model ID: `openai/whisper-small`
- Lisensi: MIT License

3. Dokumen Referensi:

- Paper: "Robust Speech Recognition via Large-Scale Weak Supervision" [1]
- Paper: "LoRA: Low-Rank Adaptation of Large Language Models" [20]
- Paper: "Exploration of Whisper Fine-Tuning Strategies for Low-Resource ASR" [2]
- Dokumentasi Hugging Face Transformers
- Dokumentasi PEFT library

3.4 Metode Pengembangan

Penelitian ini menggunakan pendekatan eksperimental dengan metodologi yang terstruktur untuk memastikan validitas dan reproducibility hasil.

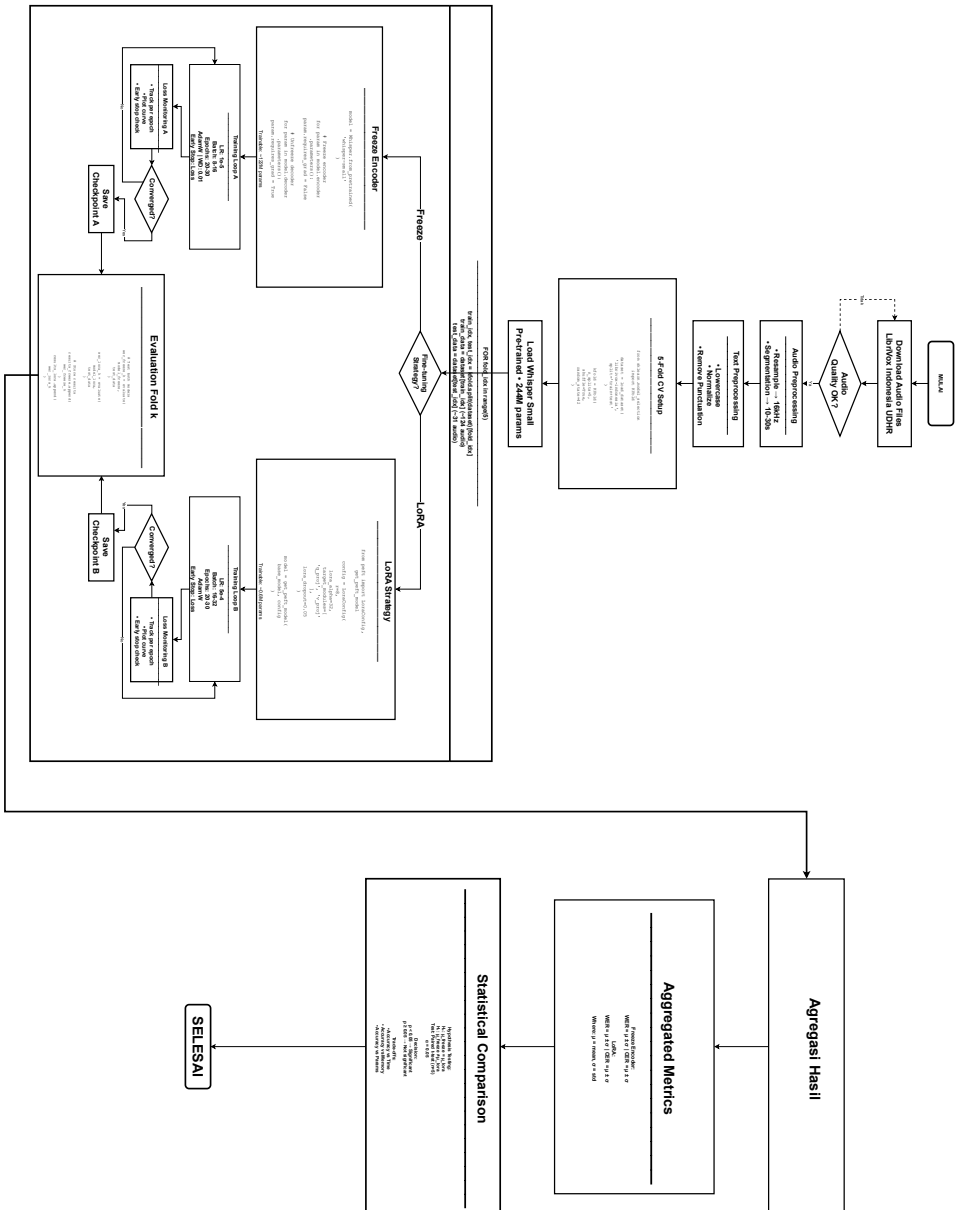
3.4.1 Pendekatan Penelitian

Penelitian ini mengadopsi **pendekatan eksperimental** dengan karakteristik:

1. **Quantitative Evaluation:** Pengukuran performa menggunakan metrik objektif (WER, CER) dengan agregasi mean $\pm$ std dari 5 folds
2. **Comparative Study:** Perbandingan antara dua strategi fine-tuning (*Freeze Encoder* vs LoRA) pada skenario extremely low-resource
3. **Controlled Experiment:** Dataset, hyperparameters, dan evaluation protocol dijaga konsisten untuk perbandingan yang adil
4. **Reproducible Research:** Semua kode, konfigurasi, dan hasil didokumentasikan dengan lengkap
5. **Statistical Validation:** Paired t-test (n=5 folds) untuk menentukan signifikansi perbedaan performa

3.4.2 Alur Pengembangan

Pengembangan sistem ASR bahasa Minangkabau mengikuti alur yang sistematis seperti digambarkan pada Gambar 3.2:



Gambar 3.2 Alur Pengembangan Sistem

3.4.3 Metodologi Fine-Tuning

Metodologi fine-tuning mengikuti best practices untuk extremely low-resource ASR dengan pendekatan 5-Fold Cross-Validation:

3.4.3.1 Freeze Encoder Fine-Tuning Methodology

Strategi ini mempertahankan representasi akustik universal dari encoder sambil mengadaptasi decoder untuk bahasa Minangkabau:

1. Initialize dari pre-trained Whisper Small checkpoint
2. **Freeze Components:** Seluruh encoder layers (12 layers) dan feature extractor tetap frozen
3. **Trainable Components:** Decoder Transformer blocks (12 layers, ~122M parameters)
4. **5-Fold Iteration:** Untuk setiap fold k ($k=1$ hingga 5):
 - Training set: 4 folds (~124 audio)
 - Test set: 1 fold (~31 audio)
 - Training dengan supervised learning dari scratch
 - Monitoring training loss untuk convergence
 - Early stopping berdasarkan loss plateau (patience=5 epochs)
5. Save best checkpoint per fold berdasarkan lowest training loss
6. Evaluate pada test fold untuk mendapatkan WER_k dan CER_k

3.4.3.2 LoRA Fine-Tuning Methodology

Parameter-Efficient Fine-Tuning dengan hanya melatih low-rank adapters (~0.24% parameters):

1. Initialize dari pre-trained Whisper Small checkpoint
2. **Freeze Components:** Seluruh base model parameters (244M params)
3. **Inject LoRA Adapters:**
 - Rank $r=8$, Alpha=16
 - Target modules: Query dan Value projections

- Trainable parameters: $\sim 0.59\text{M}$ ($\sim 0.24\%$ dari total)
4. **5-Fold Iteration:** Sama seperti Freeze Encoder, untuk setiap fold k:
 - Training pada 4 folds, test pada 1 fold
 - Higher learning rate ($5e-4$) karena parameter lebih sedikit
 - Dropout 0.1 pada LoRA layers untuk regularisasi
 5. Save LoRA adapters per fold (base model tetap frozen)
 6. Evaluate pada test fold untuk WER_k dan CER_k

3.4.3.3 Aggregation dan Statistical Testing

Setelah 5 iterasi selesai untuk kedua strategi:

1. Kumpulkan hasil: $\{\text{WER}_1, \text{WER}_2, \dots, \text{WER}_5\}$ untuk Freeze Encoder dan LoRA
2. Hitung mean $\pm$ standard deviation untuk setiap strategi
3. Paired t-test ($n=5$) untuk menguji signifikansi perbedaan: $H_0: \mu_{\text{freeze}} = \mu_{\text{lora}}$
4. Signifikansi ditentukan dengan p-value < 0.05
5. Report final metrics: “Freeze Encoder: $\text{WER} = X.X\% \pm Y.Y\%$ ”

3.4.4 Metode Pengujian

Pengujian dilakukan secara sistematis untuk memvalidasi performa model:

3.4.4.1 Functional Testing

1. **Audio Input Testing:** Verifikasi model dapat menerima berbagai format audio
2. **Transcription Testing:** Verifikasi output transkrip dalam format text yang benar
3. **Language Identification:** Verifikasi model mengenali bahasa Minangkabau dengan benar
4. **Robustness Testing:** Test dengan audio berkualitas rendah, background noise

3.4.4.2 Performance Testing

1. **Accuracy Testing:** Evaluasi WER dan CER pada test set
2. **Inference Speed Testing:** Pengukuran Real-Time Factor (RTF)
3. **Memory Usage Testing:** Monitoring GPU/CPU memory consumption
4. **Scalability Testing:** Test dengan varying input lengths

3.4.4.3 Comparative Testing

1. Perbandingan Base Whisper Small vs Fine-tuned models (mean WER/CER across 5 folds)
2. Perbandingan Freeze Encoder vs LoRA (mean $\pm$ std WER/CER dari 5 folds)
3. Statistical significance testing dengan paired t-test (n=5 folds) untuk WER dan CER
4. Analysis of variance (ANOVA) jika diperlukan untuk multiple comparisons

3.5 Ilustrasi Perhitungan Metode

Bagian ini menjelaskan contoh perhitungan untuk metrik evaluasi utama yang digunakan dalam penelitian.

3.5.1 Perhitungan Word Error Rate (WER)

WER mengukur edit distance pada level kata antara reference dan hypothesis transcription.

3.5.1.1 Formula WER

$$\text{WER} = \frac{S + D + I}{N} \times 100\% \quad (\text{Rumus 3.1})$$

di mana:

- S = number of substitutions (kata salah diganti)
- D = number of deletions (kata hilang)
- I = number of insertions (kata tambahan)

- N = total words dalam reference

3.5.1.2 Contoh Perhitungan

Misalkan kita memiliki pasangan reference-hypothesis berikut:

Reference: “*awak ingin pai ka pasar pagi iko*”

Hypothesis: “*awak ingin pergi pasar pagi ini*”

Alignment:

Reference	awak	ingin	pai	ka	pasar	pagi	iko
Hypothesis	awak	ingin	pergi	-	pasar	pagi	ini
Operation	C	C	S	D	C	C	S

Tabel 3.1 Alignment untuk perhitungan WER (C=Correct, S=Substitution, D=Deletion)

Perhitungan:

- Substitutions (S): 2 (“pai” → “pergi”, “iko” → “ini”)
- Deletions (D): 1 (“ka” dihapus)
- Insertions (I): 0
- Total words (N): 7

$$WER = \frac{2 + 1 + 0}{7} \times 100\% = \frac{3}{7} \times 100\% = 42.86\% \qquad \text{(Rumus 3.2)}$$

3.5.2 Perhitungan Character Error Rate (CER)

CER mengukur edit distance pada level karakter, lebih sensitif terhadap phonetic errors.

3.5.2.1 Formula CER

$$CER = \frac{S_c + D_c + I_c}{N_c} \times 100\% \qquad \text{(Rumus 3.3)}$$

di mana:

- S_c = number of character substitutions
- D_c = number of character deletions

- I_c = number of character insertions
- N_c = total characters dalam reference

3.5.2.2 Contoh Perhitungan

Menggunakan contoh yang sama:

Reference: “*awakinginpaikapasarpagiko*” (tanpa spasi: 30 karakter)

Hypothesis: “*awakinginpergipasarpagini*” (tanpa spasi: 28 karakter)

Perhitungan:

- Character substitutions (S_c): 5 (“p”→“p”, “a”→“e”, “i”→“r”, “k”→“g”, “o”→“i”)
- Character deletions (D_c): 2 (“k”, “a” dari kata “ka”)
- Character insertions (I_c): 0
- Total characters (N_c): 30

$$\text{CER} = \frac{5 + 2 + 0}{30} \times 100\% = \frac{7}{30} \times 100\% = 23.33\% \quad (\text{Rumus 3.4})$$

Perhatikan bahwa CER (23.33%) lebih rendah dari WER (42.86%) karena partial word matches dihargai pada level karakter.

3.5.3 Perhitungan LoRA Parameters

Perhitungan jumlah trainable parameters untuk LoRA adaptation.

3.5.3.1 Formula LoRA Parameters

Untuk setiap attention layer dengan projection weight $W \in \mathbb{R}^{d \times k}$, LoRA menambahkan:

$$\text{LoRA params} = r \times (d + k) \quad (\text{Rumus 3.5})$$

di mana r adalah rank.

3.5.3.2 Contoh Perhitungan untuk Whisper Small

Whisper Small memiliki:

- Total layers: 24 (12 encoder + 12 decoder)
- Hidden dimension (d): 768
- Projection dimension (k): 768
- LoRA rank (r): 8
- LoRA applied to: Query dan Value projections (2 per layer)

Perhitungan per layer:

$$\text{Params per projection} = 8 \times (768 + 768) = 8 \times 1536 = 12,288 \quad (\text{Rumus 3.6})$$

$$\text{Params per layer} = 2 \times 12,288 = 24,576 \quad (\text{Rumus 3.7})$$

Total LoRA parameters:

$$\text{Total LoRA params} = 24 \times 24,576 = 589,824 \approx 0.59\text{M} \quad (\text{Rumus 3.8})$$

Percentage of total parameters:

$$\text{Percentage} = \frac{589,824}{244,000,000} \times 100\% \approx 0.24\% \quad (\text{Rumus 3.9})$$

Ini menunjukkan bahwa LoRA hanya melatih $\sim 0.24\%$ dari total parameters Whisper Small, sangat efisien dibanding *Freeze Encoder*.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Penelitian

Berisi hasil penelitian berdasarkan rancangan yang sudah dijelaskan pada Bab III, terutama dari Subbab 3.4. Bagi yang membuat alat, jelaskan alat yang jadi dalam bentuk apa. Bagi yang membuat aplikasi, jelaskan aplikasi yang jadi dalam bentuk seperti apa. Jabarkan dalam bentuk pseudocode dan dijelaskan per bagian kodenya. Gunakan gambar dan tabel sebagai alat bantu menjelaskan hasil.

Contoh implementasi kode dapat ditulis menggunakan `\begin{lstlisting}`. Contoh kode dapat dilihat pada Kode 4.1.

Kode 4.1 Akuisisi Gambar

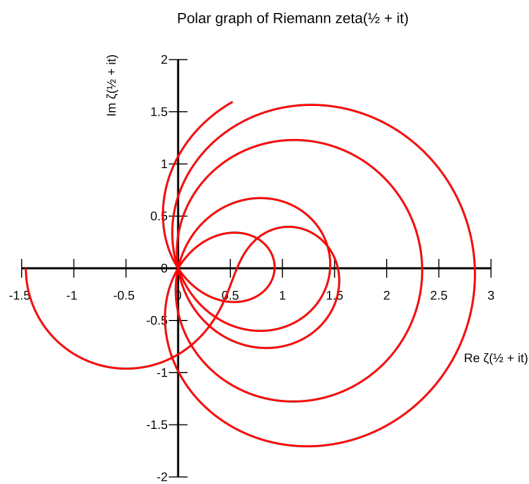
```
1 def process_dataset(dataset_path):
2     image_files = glob(os.path.join(dataset_path, '*.png'))
3     image_files.sort()
4     for image_file in image_files:
5         frame = cv2.imread(image_file)
6         if frame is None:
7             continue
8         frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
9         cv2.imshow('Frame', frame)
10        if cv2.waitKey(1) & 0xFF == ord('q'):
11            break
12    cv2.destroyAllWindows()
13 def main():
14     datasets = get_all_dataset_folders(DATASET_ROOT)
15     for dataset in datasets:
16         process_dataset(dataset)
17         print("print string")
```

4.2 Hasil Pengujian

Berikan hasil pengujian berdasarkan rancangan & skenario yang sudah direncanakan sebelumnya pada Subbab ??.

Tabel 4.1 Data *dummy* Pengujian

Subjek	Hasil Prediksi (BPM)							GT
	F	NA	NO	RC	LC	M	C	
1	68	69	68	70	68	71	69	68
2	69	69	68	70	68	71	69	69
3	70	70	69	71	68	73	69	70
4	71	70	70	72	69	73	70	71
5	72	72	70	72	70	74	70	72



Gambar 4.1 Contoh Graf Pengujian

4.3 Analisis Hasil Penelitian

Berikan analisis hasil penelitian & pengujian, berupa data yang didapatkan dari penelitian & pengujian Tugas Akhir yang sudah anda kerjakan. Gunakan gambar dan tabel sebagai alat bantu menjelaskan analisis hasil. Data luaran penelitian yang dapat dianalisis berupa:

- 1. Hasil pengujian

2. Hasil kuesioner

3. Aplikasi yang dikembangkan

Analisis dapat membandingkan dengan hasil penelitian sebelumnya yang memiliki kemiripan topik.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berisi kesimpulan dari hasil dan pembahasan terkait penelitian yang dilakukan, dapat juga berupa temuan yang Anda dapatkan setelah melakukan penelitian atau analisis terhadap tugas akhir Anda. Memberikan jawaban dari poin pada subbab Rumusan Masalah dan Tujuan Penelitian.

5.2 Saran

Berisi saran mengenai aspek tugas akhir atau temuan yang dapat dikembangkan dan diperkaya di tugas akhir selanjutnya. Saran dapat berkaitan erat pada subbab Analisis Hasil Penelitian.

DAFTAR PUSTAKA

- [1] Alec Radford et al. “Robust Speech Recognition via Large-Scale Weak Supervision”. *arXiv preprint arXiv:2212.04356* (2023). OpenAI Whisper paper.
- [2] Yunpeng Liu, Xukui Yang, and Dan Qu. “Exploration of Whisper Fine-Tuning Strategies for Low-Resource ASR”. *EURASIP Journal on Audio, Speech, and Music Processing* 2024.29 (2024).
- [3] Vincenzo Timmel et al. “Fine-tuning Whisper on Low-Resource Languages for Real-World Applications”. *arXiv preprint arXiv:2412.15726* (2024).
- [4] Zheshu Song et al. “LoRA-Whisper: Parameter-Efficient and Extensible Multilingual ASR”. *arXiv preprint arXiv:2406.06619* (2024).
- [5] Leena G. Pillai et al. “Multistage Fine-Tuning Strategies for Automatic Speech Recognition in Low-Resource Languages”. *arXiv preprint arXiv:2411.04573* (2024).
- [6] Sourav Banerjee, Ayushi Agarwal, and Promila Ghosh. “High-Precision Medical Speech Recognition through Synthetic Data and Semantic Correction: UNITED-MEDASR”. *arXiv preprint arXiv:2412.00055* (2024).
- [7] Ziyang Zhuang et al. “Towards Efficiently Whisper Fine-tuning with Monotonic Alignments”. *Proceedings of Interspeech 2025*. 2025.
- [8] Panji Arisaputra and Amalia Zahra. “Indonesian Automatic Speech Recognition with XLSR-53”. *Ingénierie des Systèmes d’Information* 27.6 (2022), pp. 973–982.
- [9] Siyu Liang and Gina-Anne Levow. “Breaking the Transcription Bottleneck: Fine-tuning ASR Models for Extremely Low-Resource

- Fieldwork Languages”. *Proceedings of the Workshop on FieldMatters (ACL 2025)*. 2025.
- [10] Ashish Vaswani et al. “Attention is All You Need”. *Advances in Neural Information Processing Systems* 30 (2017).
 - [11] Dawit Ketema Gete et al. “Whispering in Amharic: Fine-tuning Whisper for Low-Resource Language”. *arXiv preprint arXiv:2503.18485* (2025).
 - [12] Jinpeng Li et al. “Improving Whisper’s Recognition Performance for Under-Represented Language Kazakh Leveraging Unpaired Speech and Text”. *Proceedings of Interspeech 2024*. arXiv:2408.05554. 2024, pp. 2514–2518.
 - [13] Sinno Jialin Pan and Qiang Yang. “A Survey on Transfer Learning”. *IEEE Transactions on Knowledge and Data Engineering* 22.10 (2010), pp. 1345–1359.
 - [14] Alexei Baevski et al. “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations”. *NeurIPS*. 2020.
 - [15] Alexis Conneau et al. “Unsupervised Cross-Lingual Representation Learning for Speech Recognition (XLS-R)”. *arXiv preprint arXiv:2111.09296* (2021).
 - [16] Wei-Ning Hsu et al. “HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units”. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 29 (2021), pp. 3451–3460.
 - [17] Sanyuan Chen et al. “WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing”. *IEEE Journal of Selected Topics in Signal Processing* 16.6 (2022), pp. 1505–1518.
 - [18] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. *arXiv preprint arXiv:1409.0473* (2015).

- [19] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer Normalization”. *arXiv preprint arXiv:1607.06450* (2016).
- [20] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. *arXiv preprint arXiv:2106.09685* (2022).
- [21] Daniel S Park et al. “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition”. *arXiv preprint arXiv:1904.08779* (2019).
- [22] Tom Ko et al. “Audio Augmentation for Speech Recognition”. *Sixteenth Annual Conference of the International Speech Communication Association*. 2015.
- [23] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. *arXiv preprint arXiv:1711.05101* (2019).
- [24] Ron Kohavi. “A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection”. *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*. Vol. 2. Classic paper on cross-validation methodology. Montreal, Canada: Morgan Kaufmann, 1995, pp. 1137–1143.

LAMPIRAN

A Dataset

B Hasil Wawancara

C Rincian Kasus Uji